

[NLP & Text Mining]

텍스트 벡터화 – Part 2 (Text Vectorization)

박성호

컴퓨터공학부

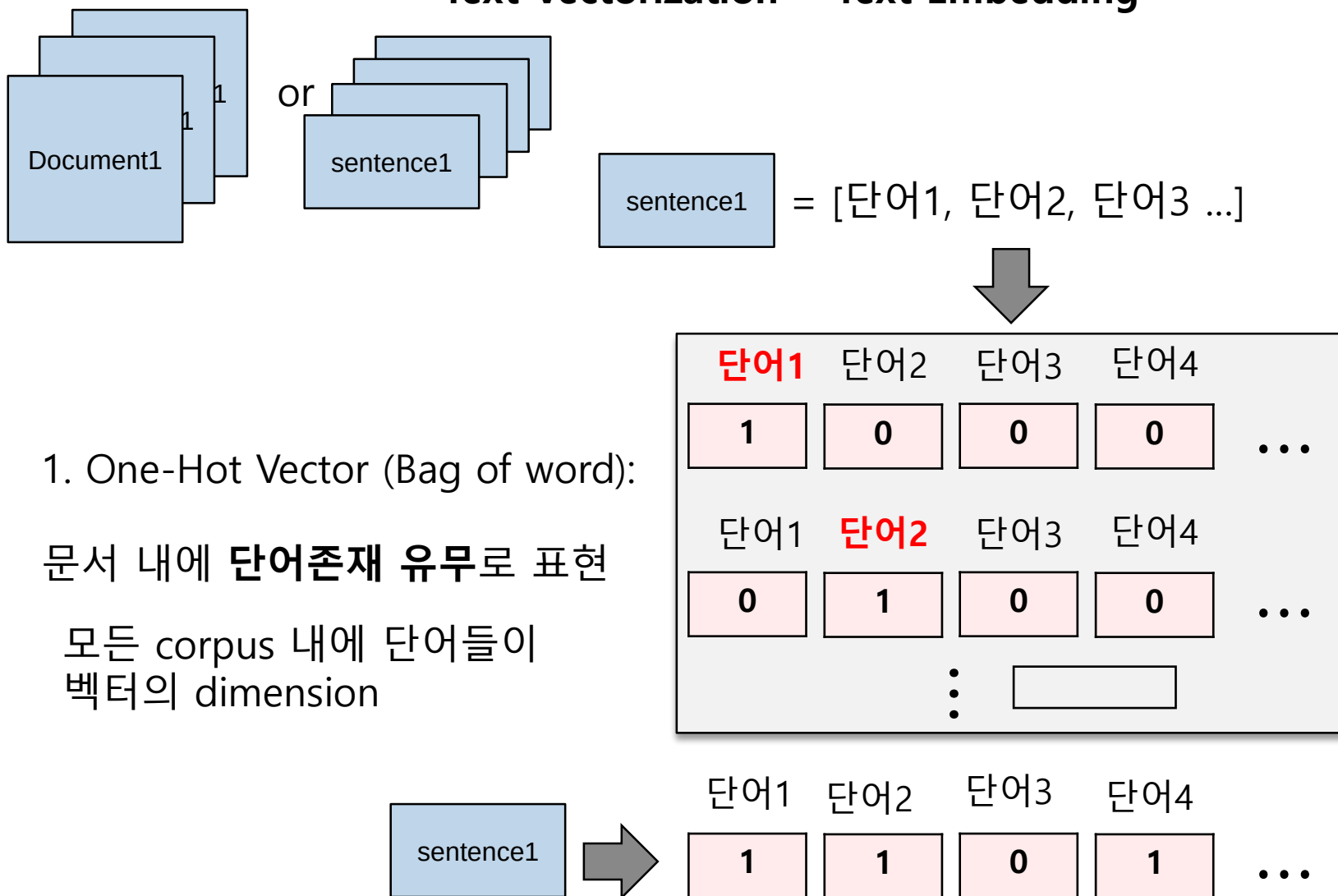
학습 목표

텍스트 벡터화에 대해 알아보겠습니다.

- Distributional Hypothesis vs Bag of Words Hypothesis
- Neural Networks 리뷰
- Word2Vec

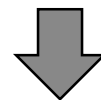
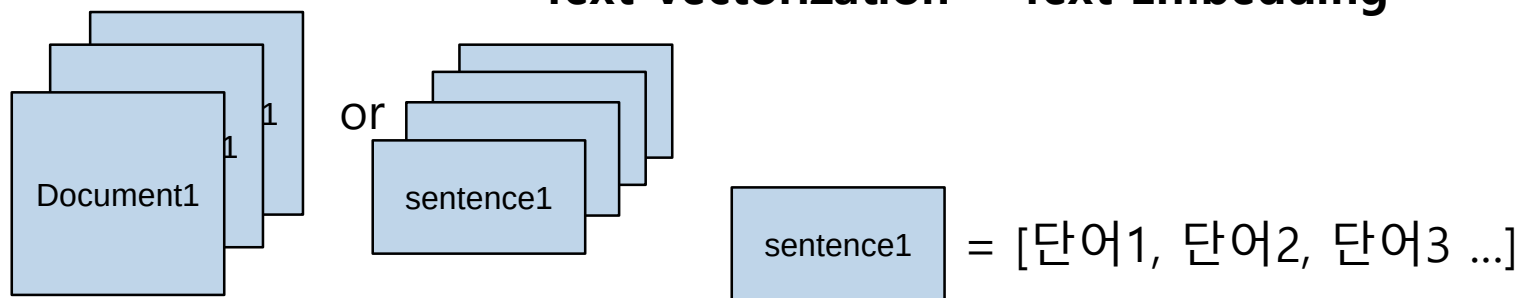
Previous – Bag of Words

Text Vectorization = Text Embedding



Previous – Bag of Words

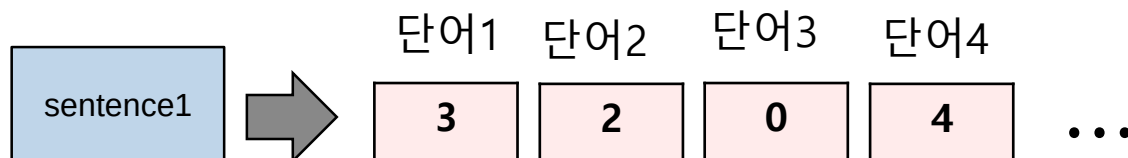
Text Vectorization = Text Embedding



2. Text Frequency (Bag of word):

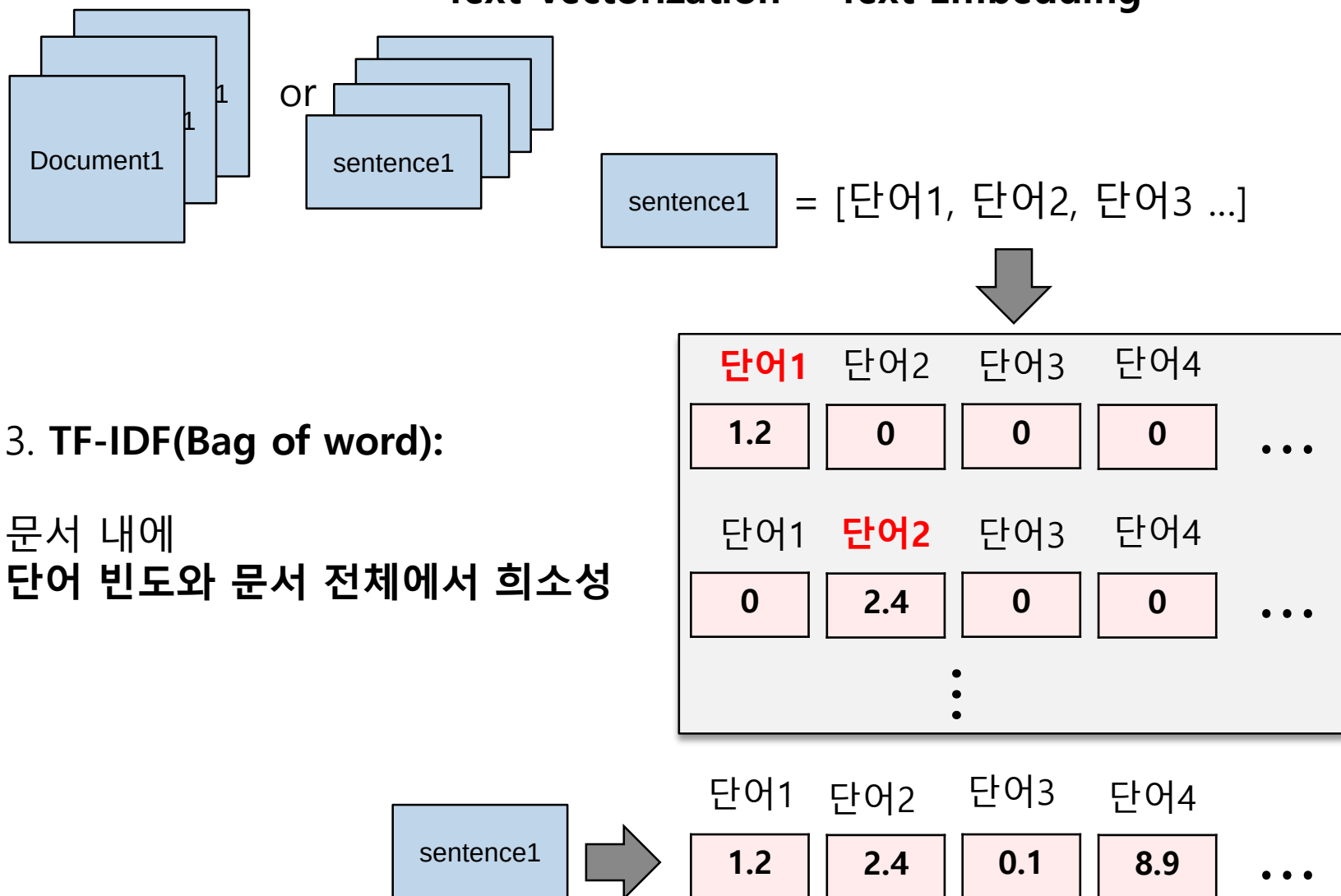
문서 내에 단어 빈도

단어1	단어2	단어3	단어4	
3	0	0	0	...
단어1	단어2	단어3	단어4	
0	2	0	0	...
				⋮



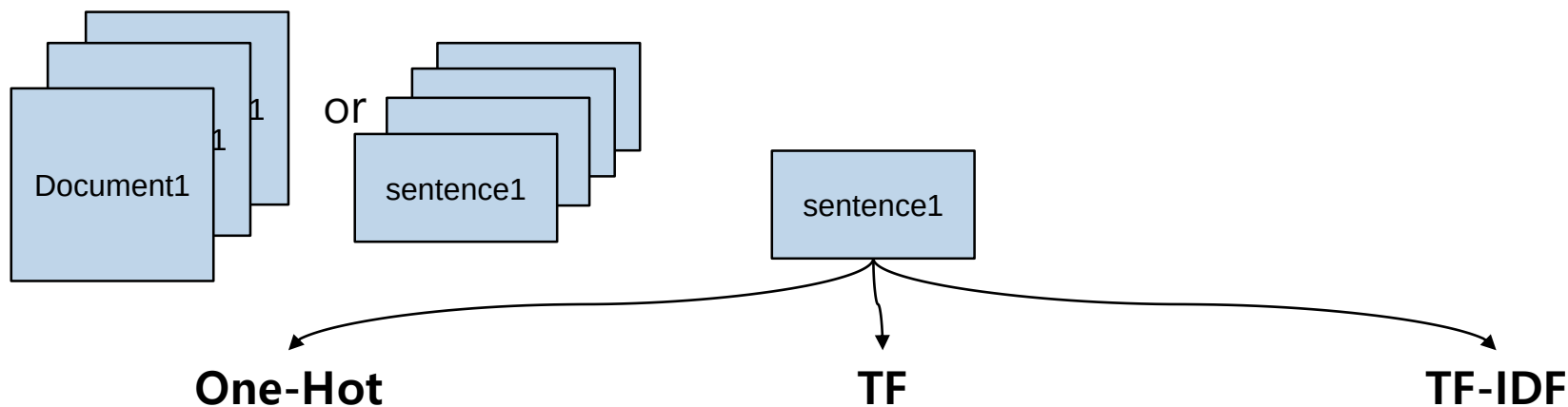
Previous – Bag of Words

Text Vectorization = Text Embedding



Previous – Bag of Words Limitations

- High Dimension – 고차원
- **Similarity between word and word (X)**-> 단어의 **문맥** 무시



고양이 = $[0,0,0,...,1,...,0] \in \mathbb{R}^{\text{백만}}$

강아지 = $[0,1,0,...,0,...,0] \in \mathbb{R}^{\text{백만}}$

애완 = $[0,0,0,...,0,...,1] \in \mathbb{R}^{\text{백만}}$

반려 = $[1,0,0,...,0,...,0] \in \mathbb{R}^{\text{백만}}$

고양이 = $[0,0,0,...,5,...,0] \in \mathbb{R}^{\text{백만}}$

강아지 = $[0,6,0,...,0,...,0] \in \mathbb{R}^{\text{백만}}$

애완 = $[0,0,0,...,0,...,7] \in \mathbb{R}^{\text{백만}}$

반려 = $[7,0,0,...,0,...,0] \in \mathbb{R}^{\text{백만}}$

고양이 = $[0,0,0,...,5.6,...,0] \in \mathbb{R}^{\text{백만}}$

강아지 = $[0,5.5,0,...,0,...,0] \in \mathbb{R}^{\text{백만}}$

애완 = $[0,0,0,...,0,...,3.5] \in \mathbb{R}^{\text{백만}}$

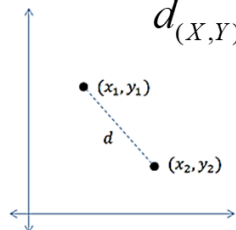
반려 = $[2.2,0,0,...,0,...,0] \in \mathbb{R}^{\text{백만}}$

Previous – Bag of Words Limitations

- High Dimension – 고차원
- **Similarity between word and word (X)->단어의 문맥 무시**

유사도 Similarity

❖ 유클리디안 거리 (Euclidean Distance)


$$d_{(X,Y)} = \sqrt{\sum_{i=1}^p (x_i - y_i)^2}$$
$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

❖ Cosine 유사도



$$A \cdot B = |A| * |B| * \cos \Theta$$
$$\cos(\Theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i \times B_i}{\sqrt{\sum_{i=1}^n (A_i)^2} \times \sqrt{\sum_{i=1}^n (B_i)^2}}$$

고양이 = [0,0,0,...1,...0] ∈ ℝ백만

철강 = [0,0,0,...1,...0] ∈ ℝ백만

강아지 = [0,1,0,...0,...0] ∈ ℝ백만

철강 = [0,0,0,...1,...0] ∈ ℝ백만

고양이 = [0,0,0,...1,...0] ∈ ℝ백만

강아지 = [0,1,0,...0,...0] ∈ ℝ백만

Distributional Hypothesis – Word Vectorization (Embedding)

- The purpose of word embedding is to **map the words** in a language **into a vector space** so that **semantically similar words are located close to each other**.

- Word vectors: distributed representation

- ✓ A parameterized function mapping words in some language to a certain dimensional vectors

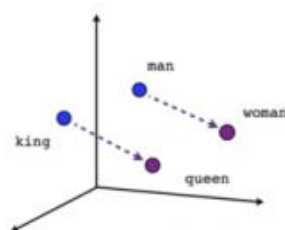
$$W : \text{words} \rightarrow \mathbb{R}^n$$

$$W(\text{"cat"}) = (0.2, -0.4, 0.7, \dots)$$

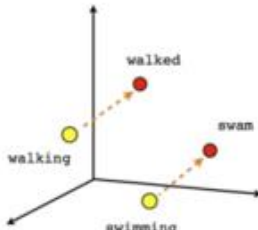
$$W(\text{"mat"}) = (0.0, 0.6, -0.1, \dots)$$

- Interesting feature of word embedding

- ✓ Semantic relationship between words can be preserved



Male-Female



Verb tense



Country-Capital

Distributional Hypothesis – Word Vectorization (Embedding)

아이디어: 단어의 의미는 주변 단어 (문맥; Context)에 의해 형성된다.

- I drink beer
- we drink wine
- I drink beer
- we guzzle (폭음하다) wine

Distributional Hypothesis – Word Vectorization (Embedding)

아이디어: 단어의 의미는 주변 단어 (문맥; Context)에 의해 형성된다.

- 과거: 통계 이론 기반의 방법론 활용
- 최근: **Neural Network** 기반의 방법론 사용 -> 딥러닝 + 하드웨어 기술 발달 GPU

1. Word2Vec

- Continuous bag-of-words (CBOW)
- Skip-gram

2. GloVe

3. Fasttext

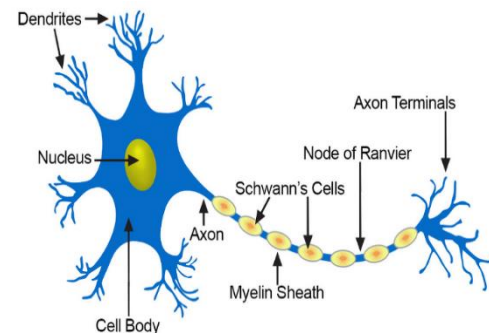
4. 기타 등등

Neural Networks Reviews

Neural Networks-Perceptron

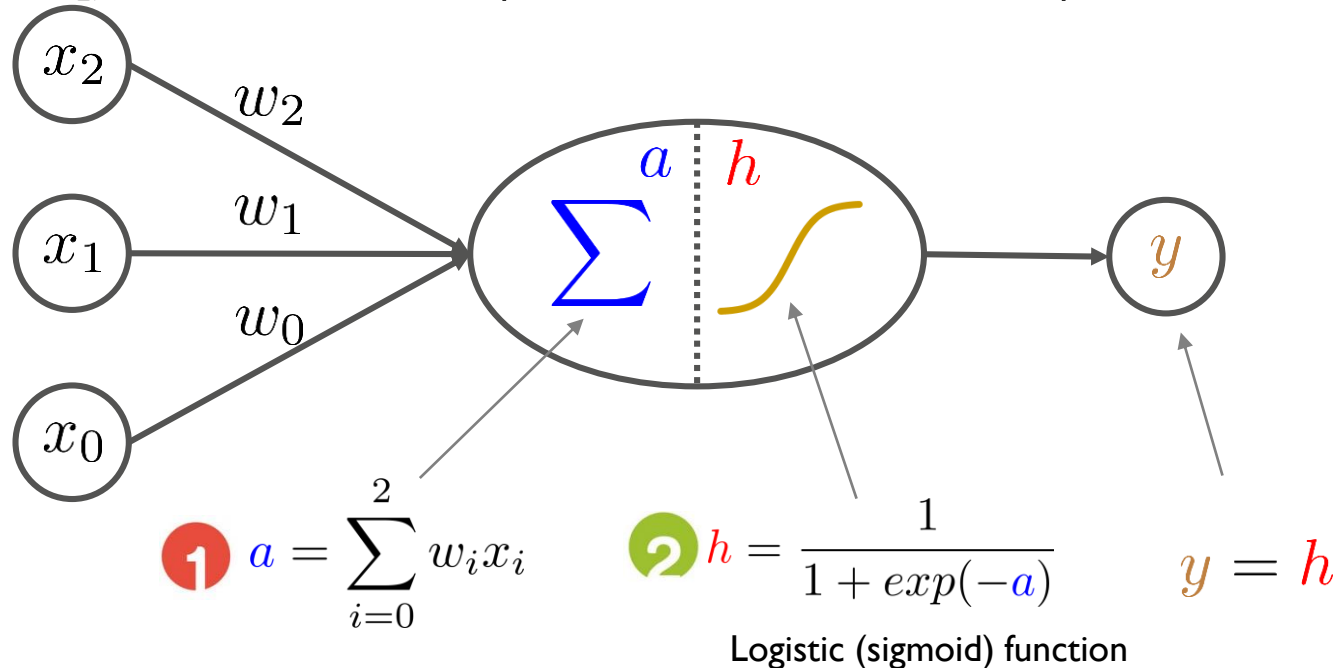
❖ 하나의 Logistic Regression 모델 (Perceptron)은 하나의 뉴론에 상응한다.

단일뉴런



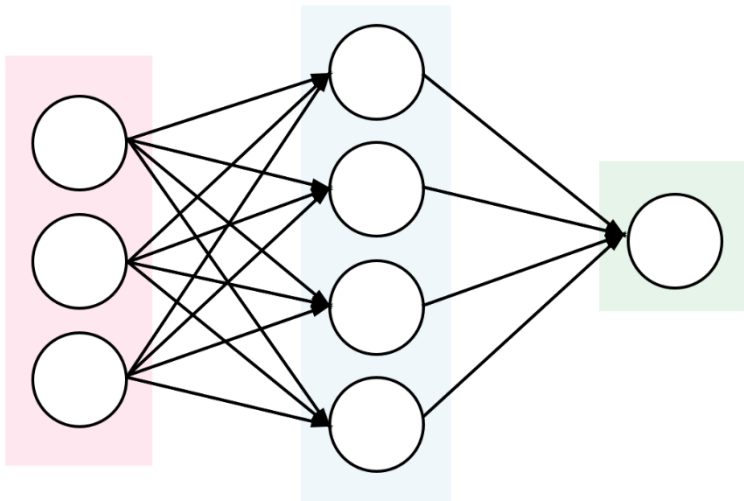
❶ 입력변수의 선형결합

❷ 선형결합 값의 비선형 변환 (Nonlinear transformation)



Neural Networks-Multi-Layer Perceptron

- 다층 퍼셉트론 → 인공신경망 (Artificial Neural Networks)

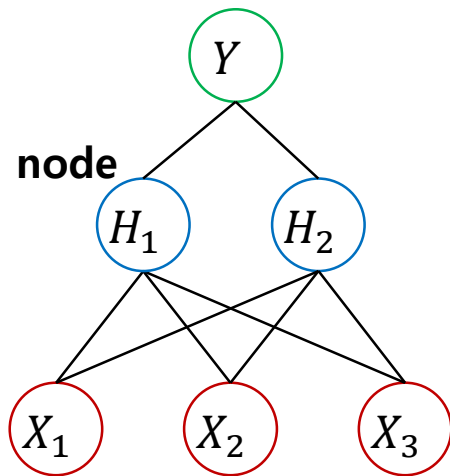


Source: <https://blog.goodaudience.com/artificial-neural-networks-explained-436fcf36e75>

- 입력층(Input Layer)
 - 입력변수의 값이 들어오는 곳
 - 입력변수의 수 = 입력노드의 수
- 은닉층(Hidden Layer)
 - 은닉층에는 다수 노드 포함 가능
 - 다수의 은닉층을 형성 가능
- 출력층 (Output Layer)
 - [범주형] 출력노드의 수 = 출력 변수의 범주 개수
 - [연속형] 출력노드의 수 = 출력 변수의 개수

Neural Networks

- 다층 퍼셉트론 → 인공신경망 (Artificial Neural Networks)
- 정보의 **선형/비선형 변환들의 조합**으로 최종 아웃풋을 도출하는 모델



Output layer

$$Y_i = \text{Activation} \left(W_{1i}^{(2)} H_1 + W_{2i}^{(2)} H_2 + b^{(2)} \right)$$

Hidden layer

$$H_i = \text{Activation} \left(W_{1i}^{(1)} X_1 + W_{2i}^{(1)} X_2 + W_{3i}^{(1)} X_3 + b_i^{(1)} \right)$$

Input layer

W : 뉴럴네트워크 모델의 파라미터

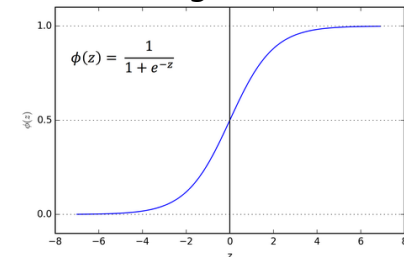
- 파라미터
 - 층 간 노드를 연결하는 가중치 ($w_{11}, w_{12}, \dots, z_{11}$) → 알고리즘으로 결정
- 하이퍼파라미터
 - 은닉층 개수, 은닉노드 개수, activation function → 사용자가 임의로 결정

Neural Networks – Activation Function

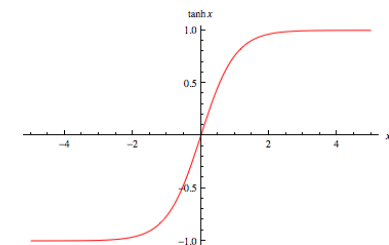
- 정보의 **선형/비선형 변환들의 조합**으로 최종 아웃풋을 도출하는 모델
- 입력변수의 비선형 변환으로 새로운 변수를 재탄생 시키는 과정

Name	Plot	Equation	Derivative (with respect to x)
Identity		$f(x) = x$	$f'(x) = 1$
Binary step		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x \neq 0 \\ ? & \text{for } x = 0 \end{cases}$
Logistic (a.k.a. Soft step)		$f(x) = \frac{1}{1 + e^{-x}}$	$f'(x) = f(x)(1 - f(x))$
TanH		$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$
ArcTan		$f(x) = \tan^{-1}(x)$	$f'(x) = \frac{1}{x^2 + 1}$
Softsign [7][8]		$f(x) = \frac{x}{1 + x }$	$f'(x) = \frac{1}{(1 + x)^2}$
Rectified linear unit (ReLU)[9]		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Leaky rectified linear unit (Leaky ReLU)[10]		$f(x) = \begin{cases} 0.01x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0.01 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Parametric rectified linear unit (PReLU)[11]		$f(\alpha, x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(\alpha, x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Randomized leaky rectified linear unit (RRReLU)[12]		$f(\alpha, x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(\alpha, x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Exponential linear unit (ELU)[13]		$f(\alpha, x) = \begin{cases} \alpha(e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(\alpha, x) = \begin{cases} f(x) + \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$

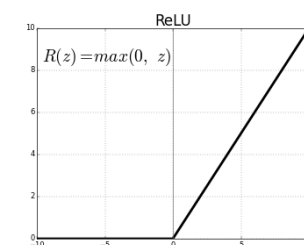
<Sigmoid>



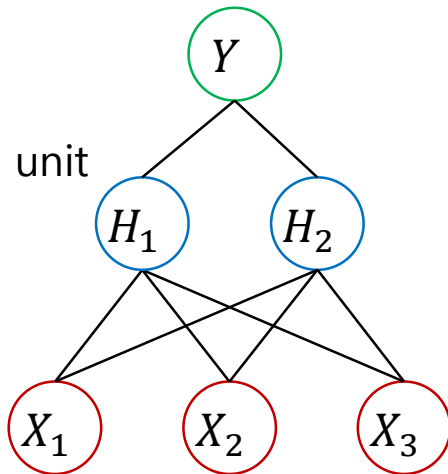
<tanh>



<relu>



Neural Networks – Activation Function



Output layer

$$Y_i = \text{Activation} \left(W_{1i}^{(2)} H_1 + W_{2i}^{(2)} H_2 + b^{(2)} \right)$$

Hidden layer

$$H_i = \text{Activation} \left(W_{1i}^{(1)} X_1 + W_{2i}^{(1)} X_2 + W_{3i}^{(1)} X_3 + b_i^{(1)} \right)$$

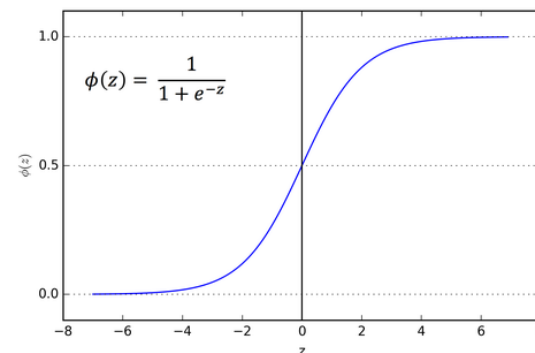
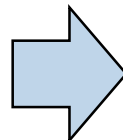
Input layer

W : 뉴럴네트워크 모델의 파라미터

Activation Function: Output Layer

- Two-class classification: $\text{sigmoid}(x) = \frac{1}{1+e^{-x}}$... sigmoid function

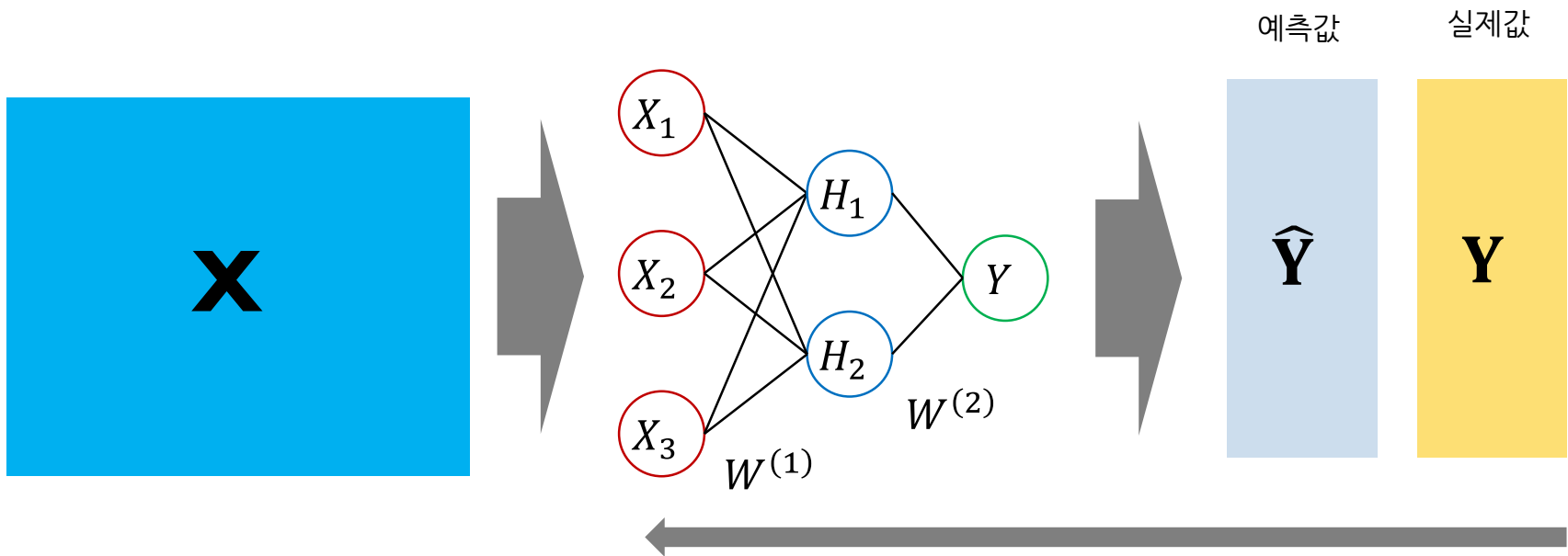
$$Y_i = \frac{1}{1 + e^{-(W_{1i}^{(2)} H_1 + W_{2i}^{(2)} H_2 + b^{(2)})}}$$



뉴럴네트워크 파라미터 w 결정, 비용(목적, 손실)함수

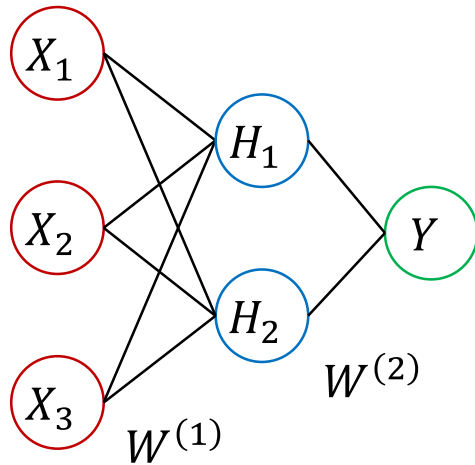
- 뉴럴네트워크 모델로부터 나온 $\hat{Y}$ 값과 실제 Y 값의 차이를 최소로 하는 가중치

$$\arg \min_w \sum_i L(Y_i, f(X_i; w))$$

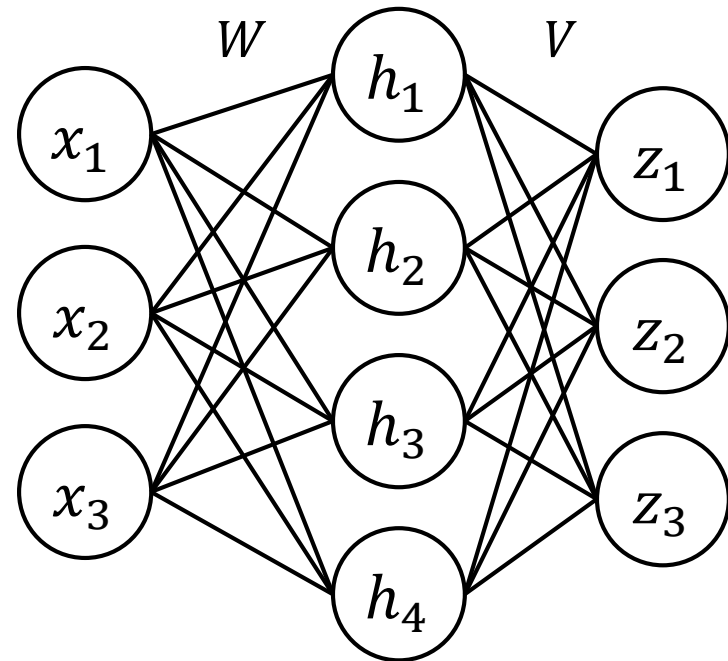


다중 클래스 – Softmax Function

Two-class Output node=1 or 2

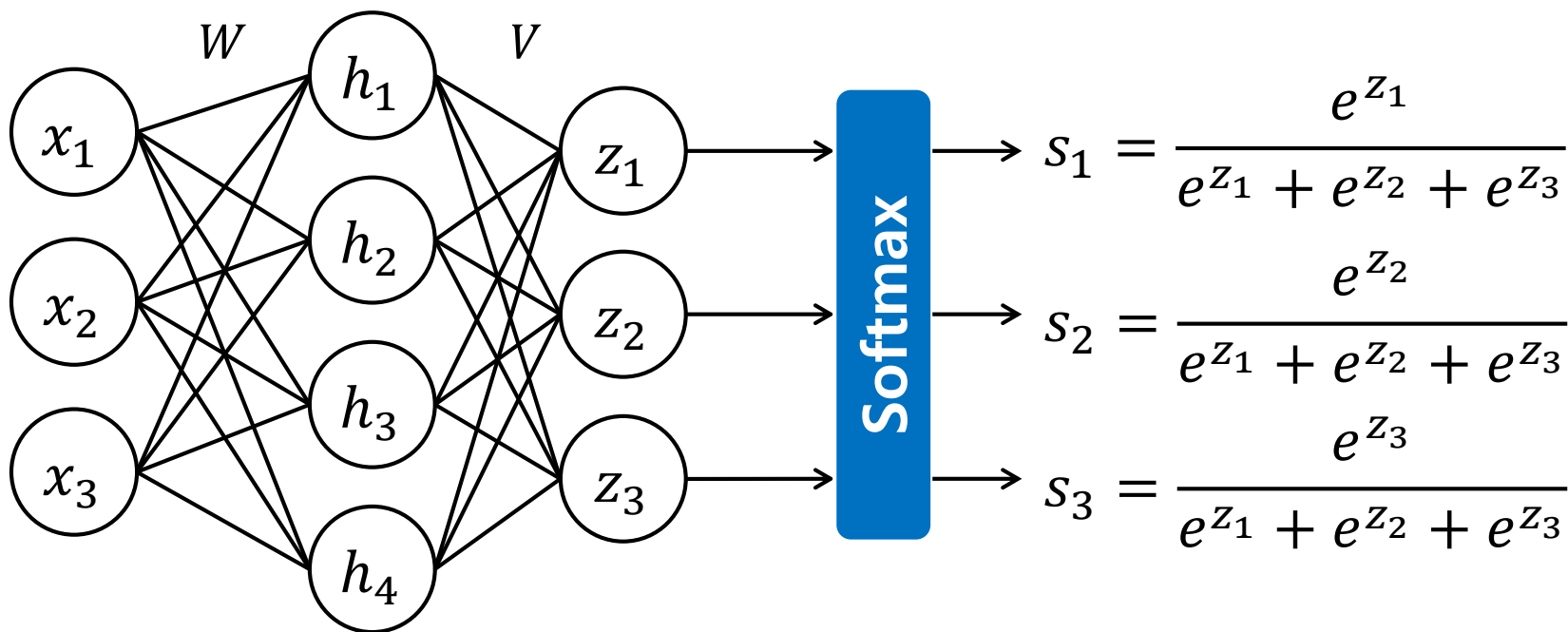


Multi-class Output node

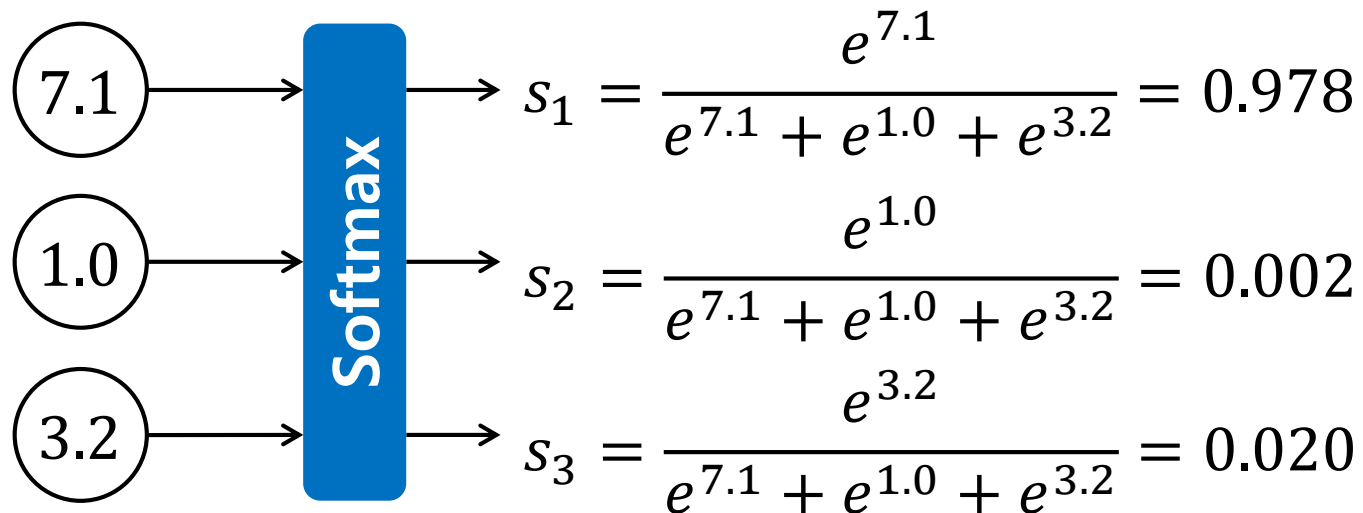


다중 클래스 – Softmax Function

- ❖ 지금까지는 output layer에서 sigmoid function을 사용하여 이진 분류 수행
- ❖ Output layer의 크기를 범주 개수만큼 설정하고 softmax를 사용하면 다중 범주 분류 (multiclass classification) 모델로 손쉽게 변형 가능



다중 클래스 – Softmax Function



$$p(y = i|x) = s_i = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad i = 1, 2, \dots, C.$$

다중 클래스 – Softmax Function

- Classification: cross entropy
- $\sum_i L(Y_i, f(X_i; w))$
- $L(y_c, \hat{y}_c) = -\sum_c y_c \log \hat{y}_c$ (y_c :실제값, $\hat{y}_c$:예측값)

예측값			실제값		
Class1	Class2	Class3	Class1	Class2	Class3
0.3	0.2	0.5	1	0	0
0.1	0.8	0.1	0	1	0
0.6	0.2	0.2	1	0	0
0.1	0.5	0.4	0	0	1

$$\sum_i L(Y_i, \hat{y}^i) = -[\ln(0.3) + \ln(0.8) + \ln(0.6) + \ln(0.4)] = 2.85$$

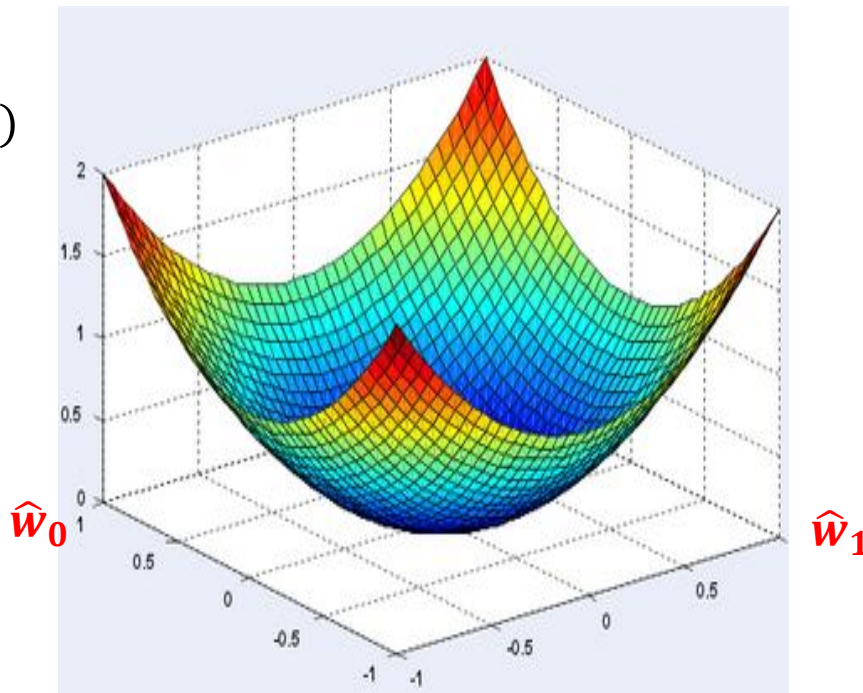
$$\arg \min_w \sum_i L(Y_i, \hat{y}^i)$$

Neural Networks Learning – 경사하강법 Gradient Descent

$$\arg \min_w \sum_i L(Y_i, \mathbf{f}(X_i; w))$$

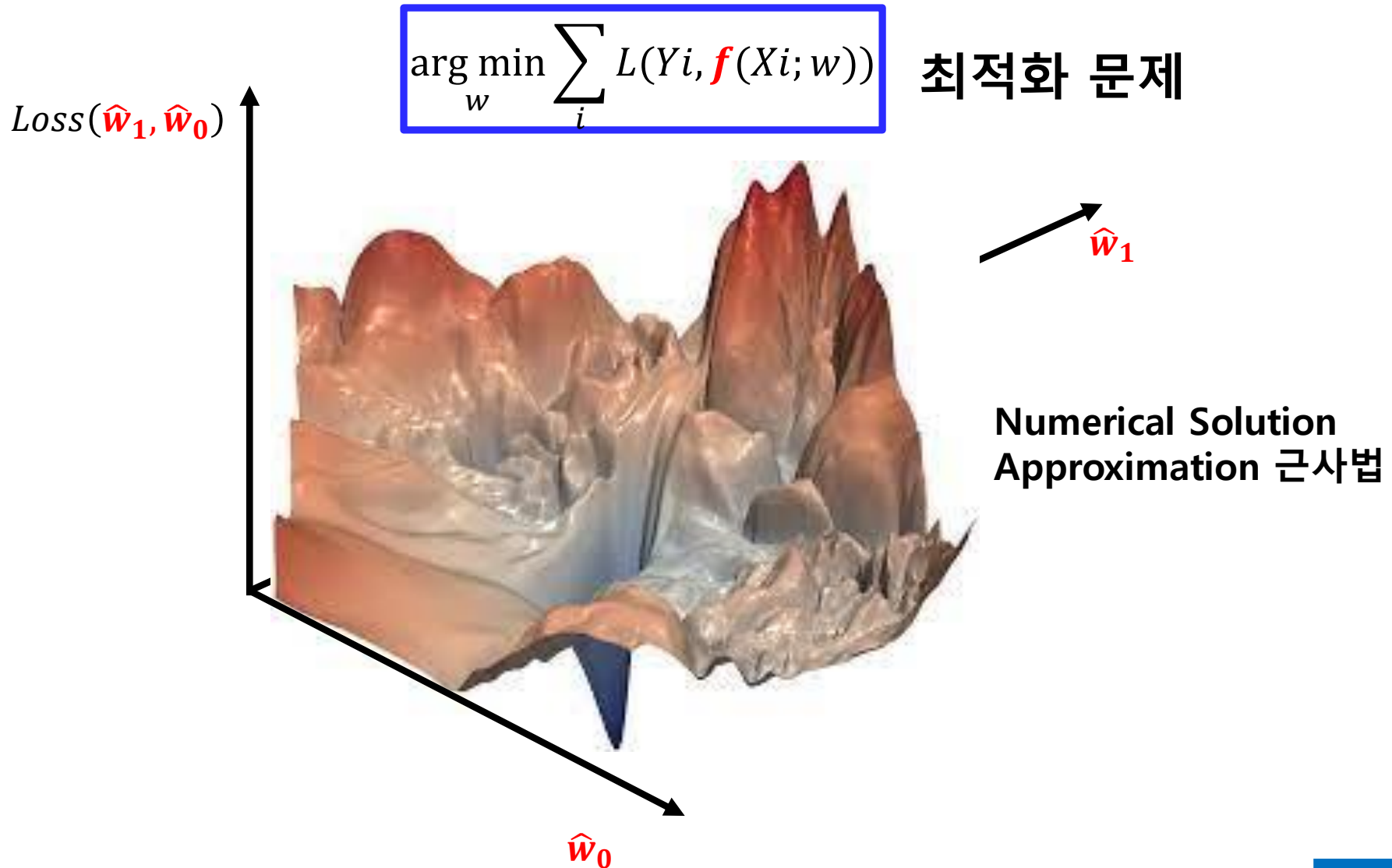
최적화 문제

$Loss(\hat{w}_1, \hat{w}_0)$



Closed-Form Solution

Neural Networks Learning – 경사하강법 Gradient Descent



Neural Networks Learning – 경사하강법 Gradient Descent

First-order optimization methods (Gradient Decent)

$$\hat{\mathbf{w}}(k+1) := \hat{\mathbf{w}}(k) - \alpha \cdot \text{Loss}^{(1)}(\hat{\mathbf{w}}(k))$$

$$\text{Loss}(\hat{\mathbf{w}}(k)) = \sum_i L(Y_i, f(X_i; \mathbf{w}))$$

$\hat{\mathbf{w}}(0) = \text{random}()$

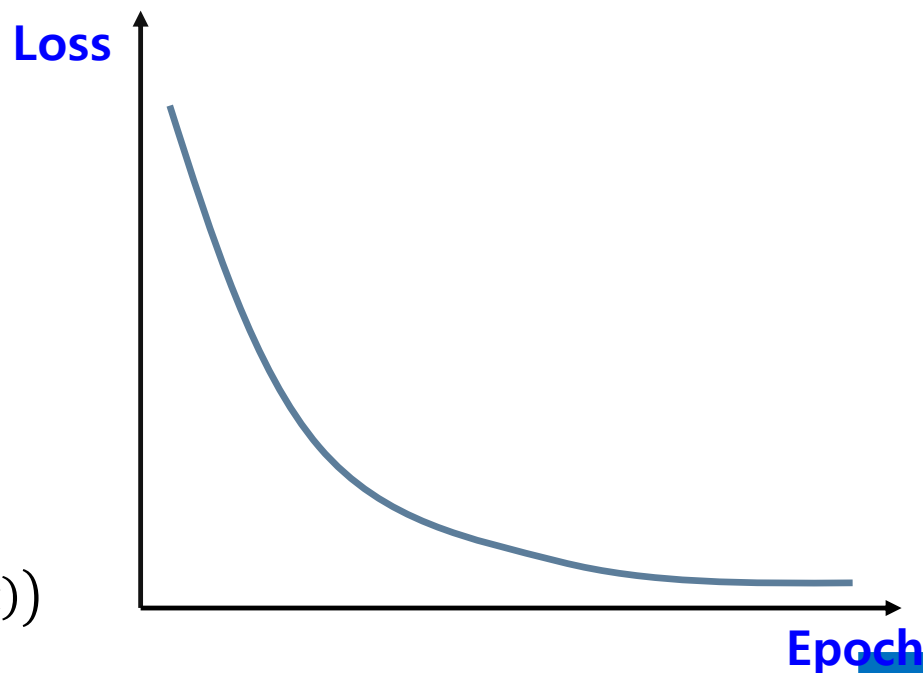
$\alpha = 0.001$

Epoch=1000 (반복 횟수)

For k in range(0, Epoch)

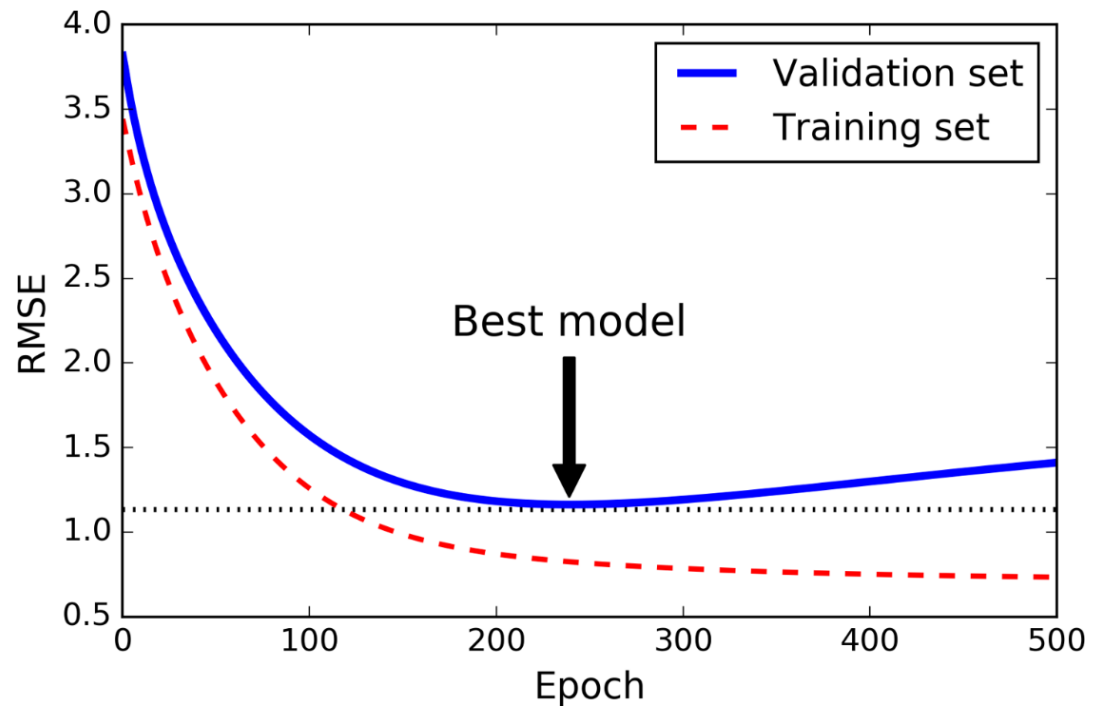
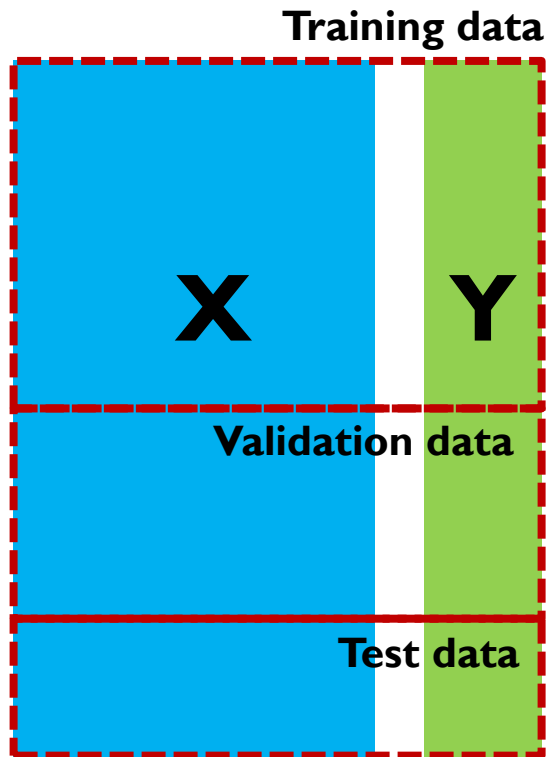
$\text{Loss}^{(1)}(\hat{\mathbf{w}}(k))$

$\hat{\mathbf{w}}(k+1) := \hat{\mathbf{w}}(k) - \alpha \cdot \text{Loss}^{(1)}(\hat{\mathbf{w}}(k))$

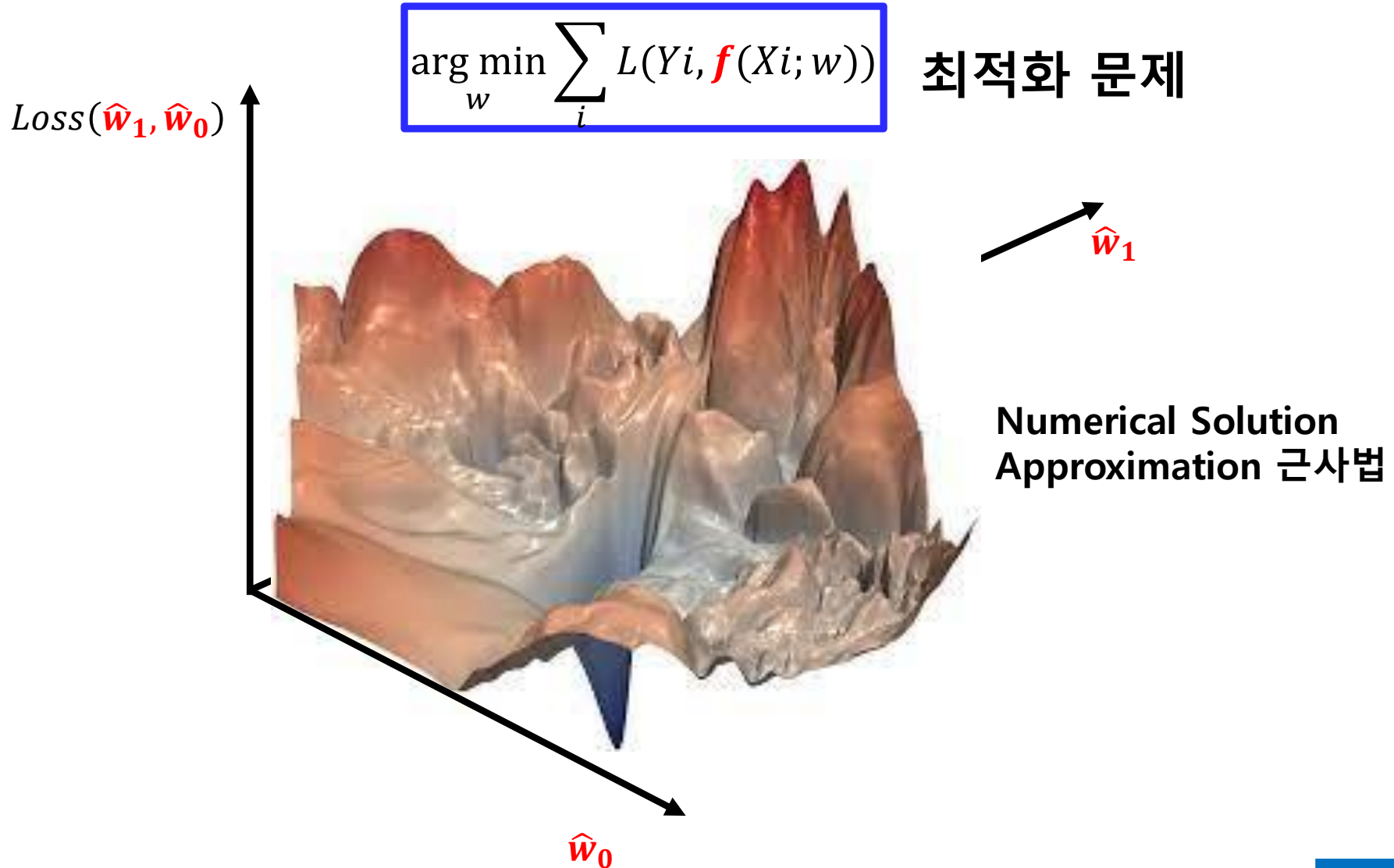


Early Stopping

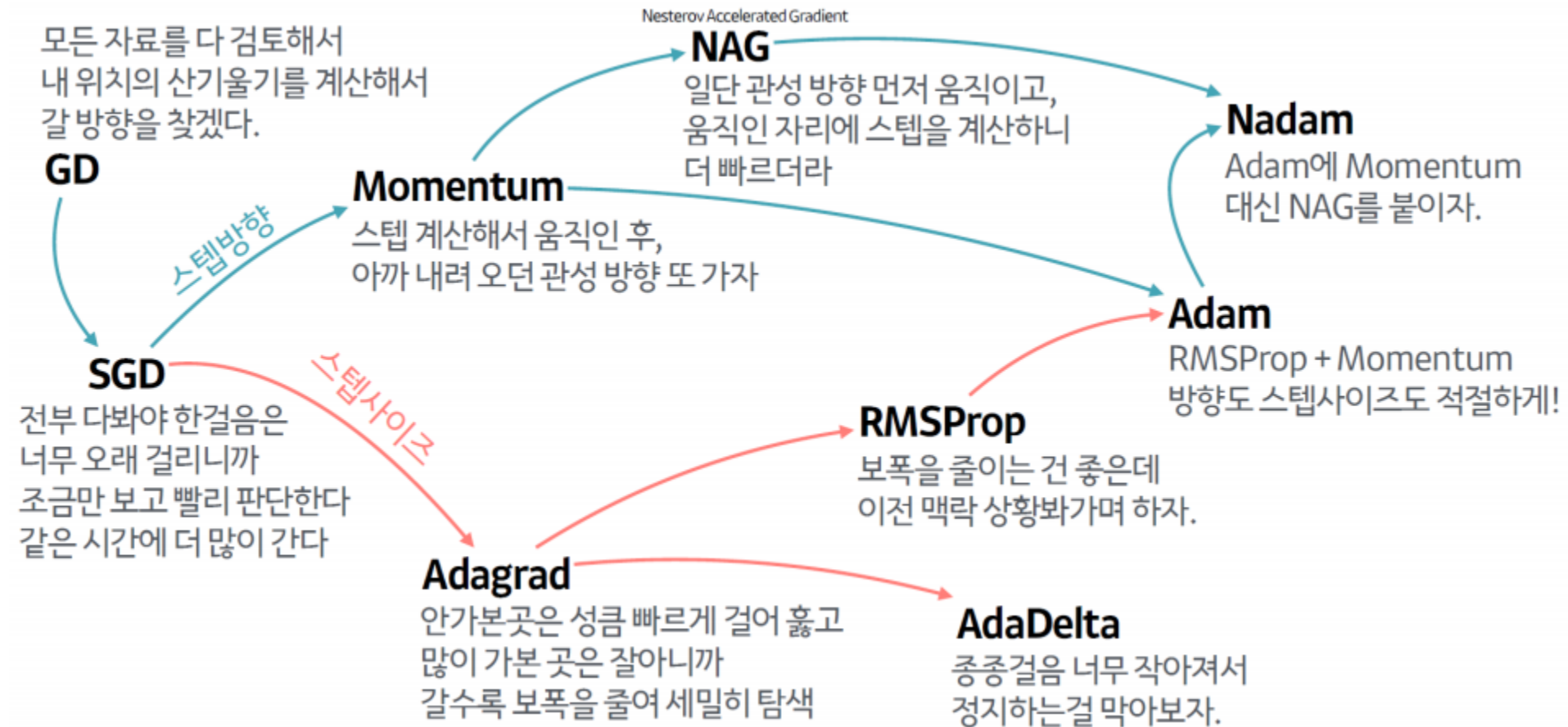
- ❖ 학습 과정을 모니터링하여 조기에 학습을 종료 (overfitting 방지)



Neural Networks Learning – 경사하강법 Gradient Descent



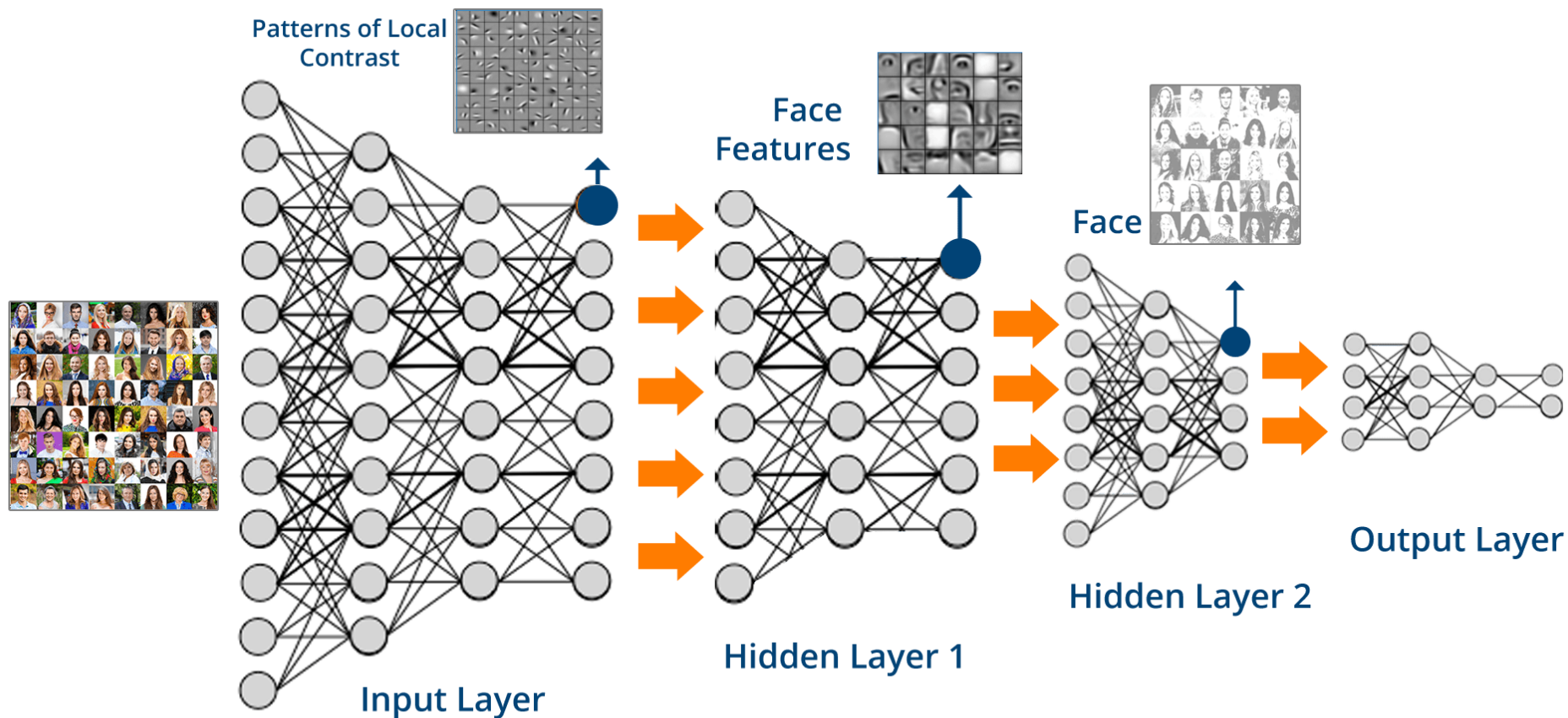
• 다양한 경사하강법



하용호. 자습해도 모르겠던 딥러닝, 머리속에 인스톨 시켜드립니다. (<https://www.slideshare.net/yongho/ss-79607172>)

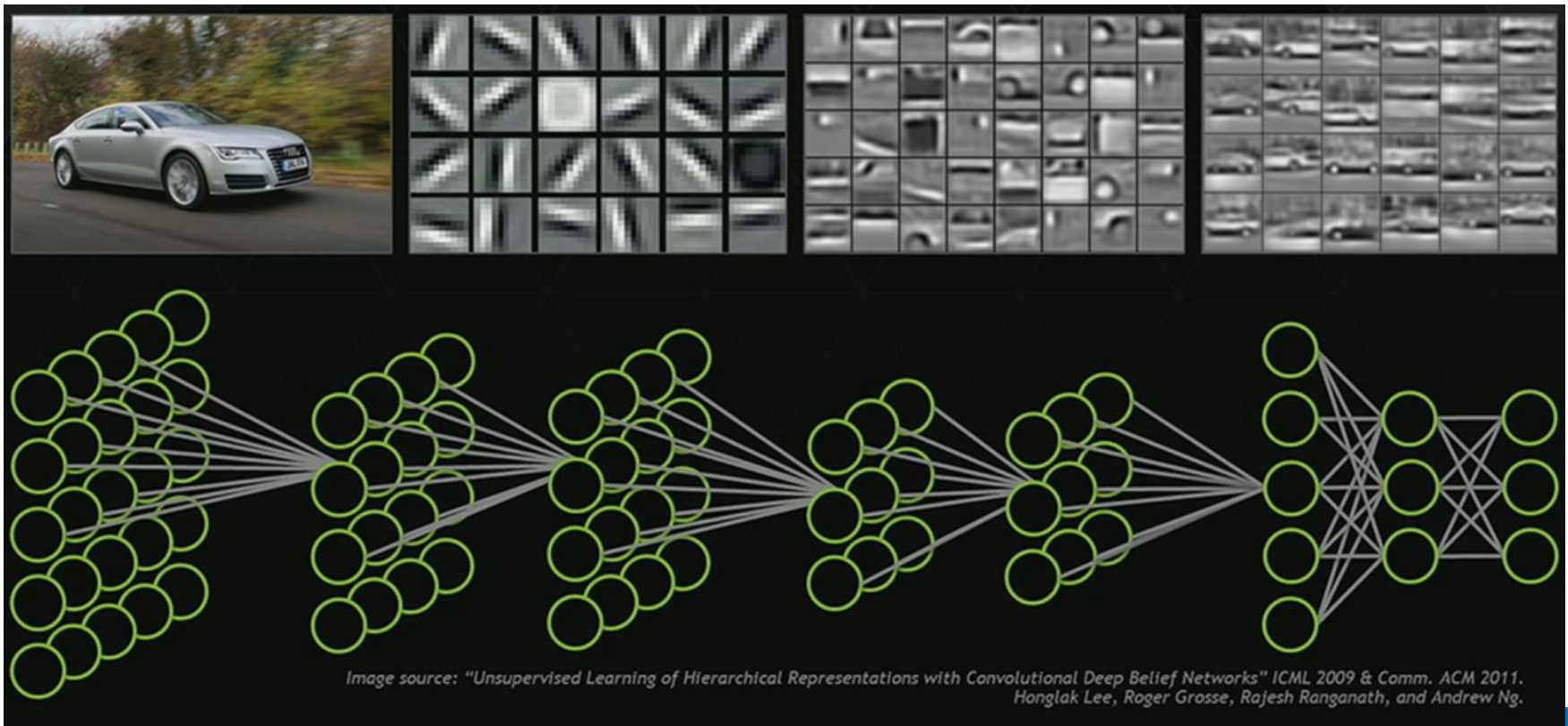
Deep Neural Network (DNN, 심층 신경망)

- Deep Neural Network (DNN)
 - 여러 층으로 이루어진 복잡한 구조의 인공신경망



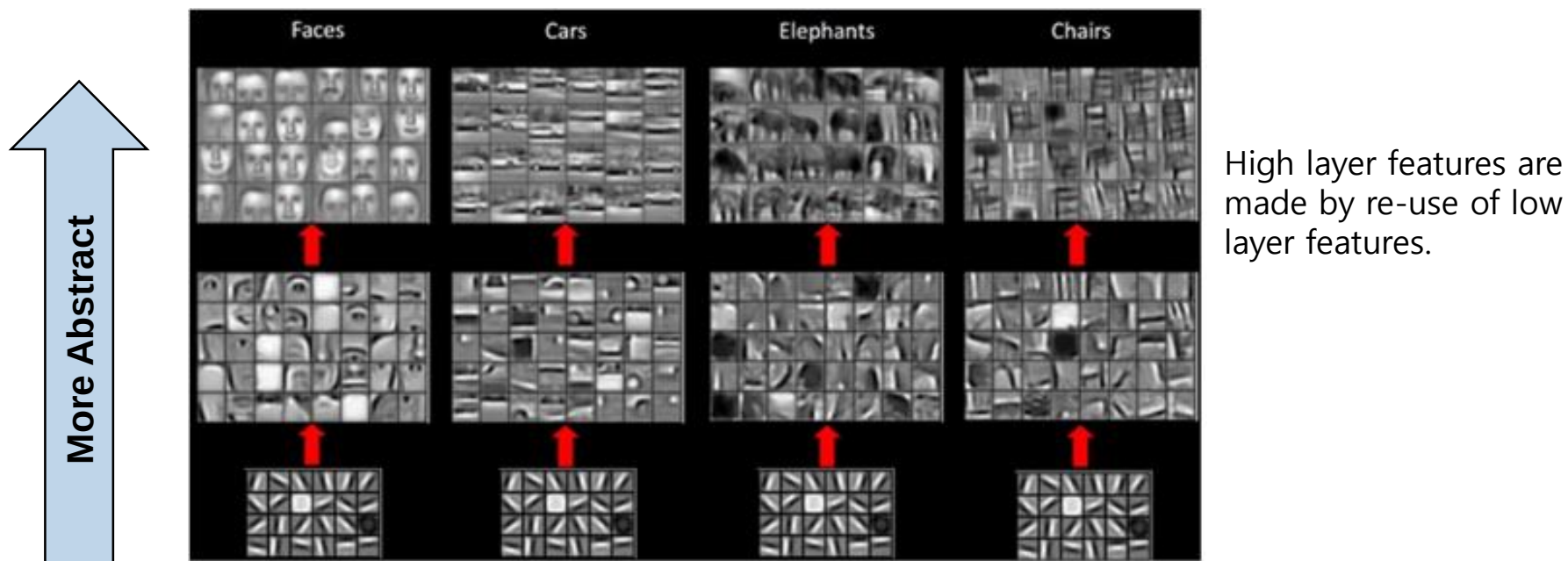
Deep Neural Network (DNN, 심층 신경망)

- Key advantage of DNN – Representation Learning
 - Each neuron must represent something, so this must be local representation.
 - Deep architectures can potentially lead to more **abstract features** at higher of representation.



Deep Neural Network (DNN, 심층 신경망)

- Key advantage of DNN– Representation Learning
 - Each neuron must represent something, so this must be local representation.
 - Deep architectures can potentially lead to more **abstract features** at higher of representation.
 - More abstract features are generally **invariant** to most local changes of the input.



Deep Neural Network (DNN, 심층 신경망)

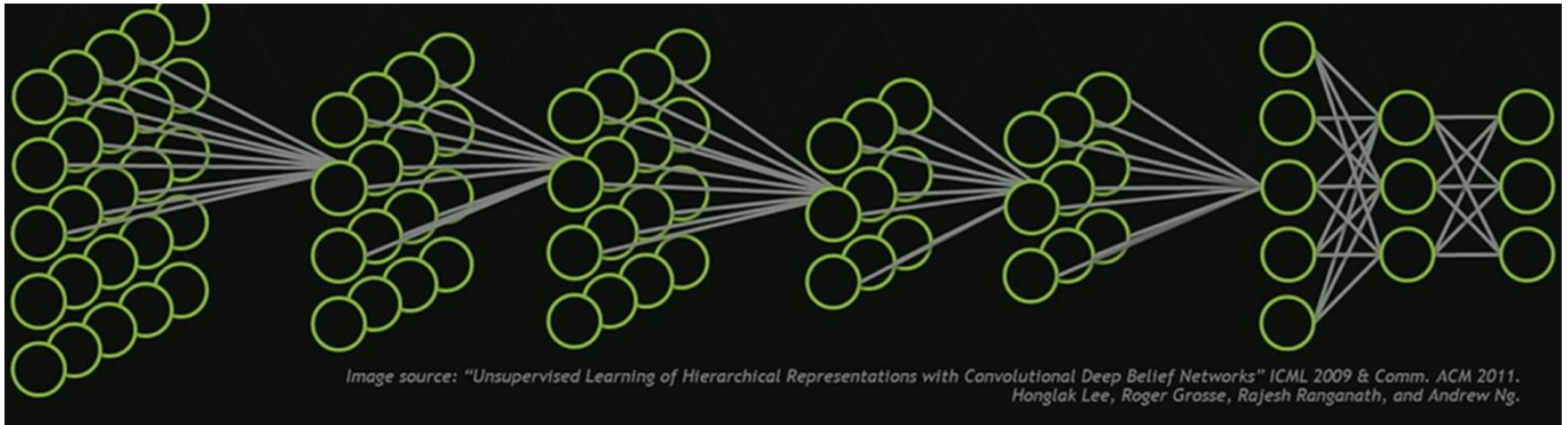
- Key advantage of DNN– Representation Learning

Corpus
문장

토큰

문맥, 어절

문장 의미



Distributional Hypothesis – Word Vectorization (Embedding)

- The purpose of word embedding is to **map the words** in a language **into a vector space** so that **semantically similar words are located close to each other**.

- Word vectors: distributed representation

- ✓ A parameterized function mapping words in some language to a certain dimensional vectors

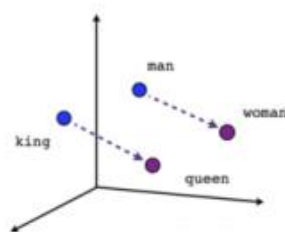
$$W : \text{words} \rightarrow \mathbb{R}^n$$

$$W(\text{"cat"}) = (0.2, -0.4, 0.7, \dots)$$

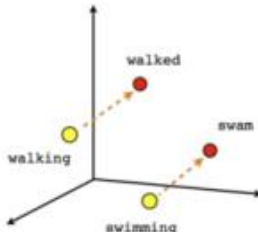
$$W(\text{"mat"}) = (0.0, 0.6, -0.1, \dots)$$

- Interesting feature of word embedding

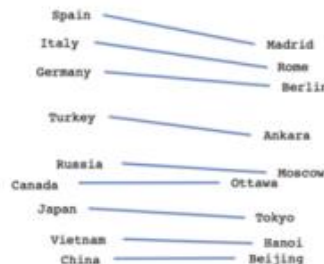
- ✓ Semantic relationship between words can be preserved



Male-Female

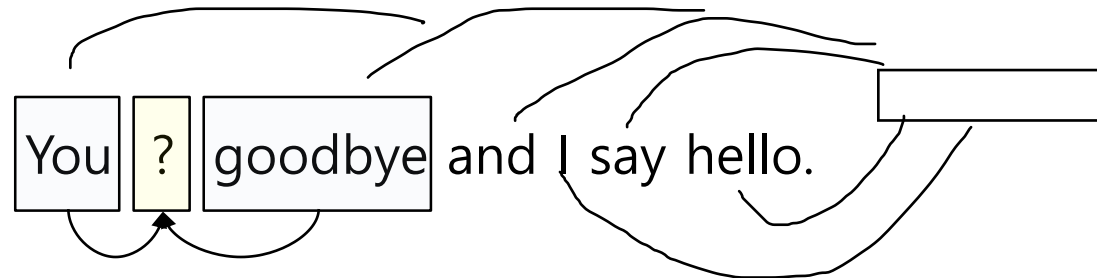


Verb tense

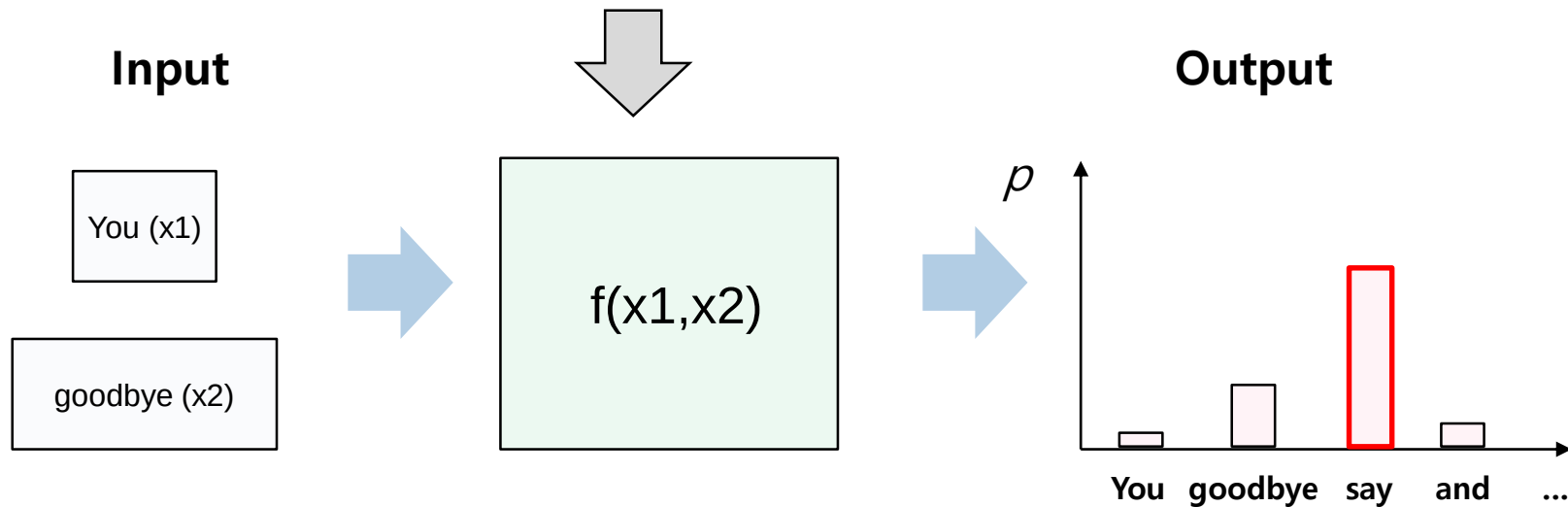


Country-Capital

Word2Vec - CBOW



- 주변에 맥락을 Input으로 받아서 ? 부분에 적합한 단어를 Output으로 산출하는 NN 모델을 학습 및 활용
- 맥락정보를 받아서 각 단어의 출현 확률을 출력하는 NN



Word2Vec - CBOW

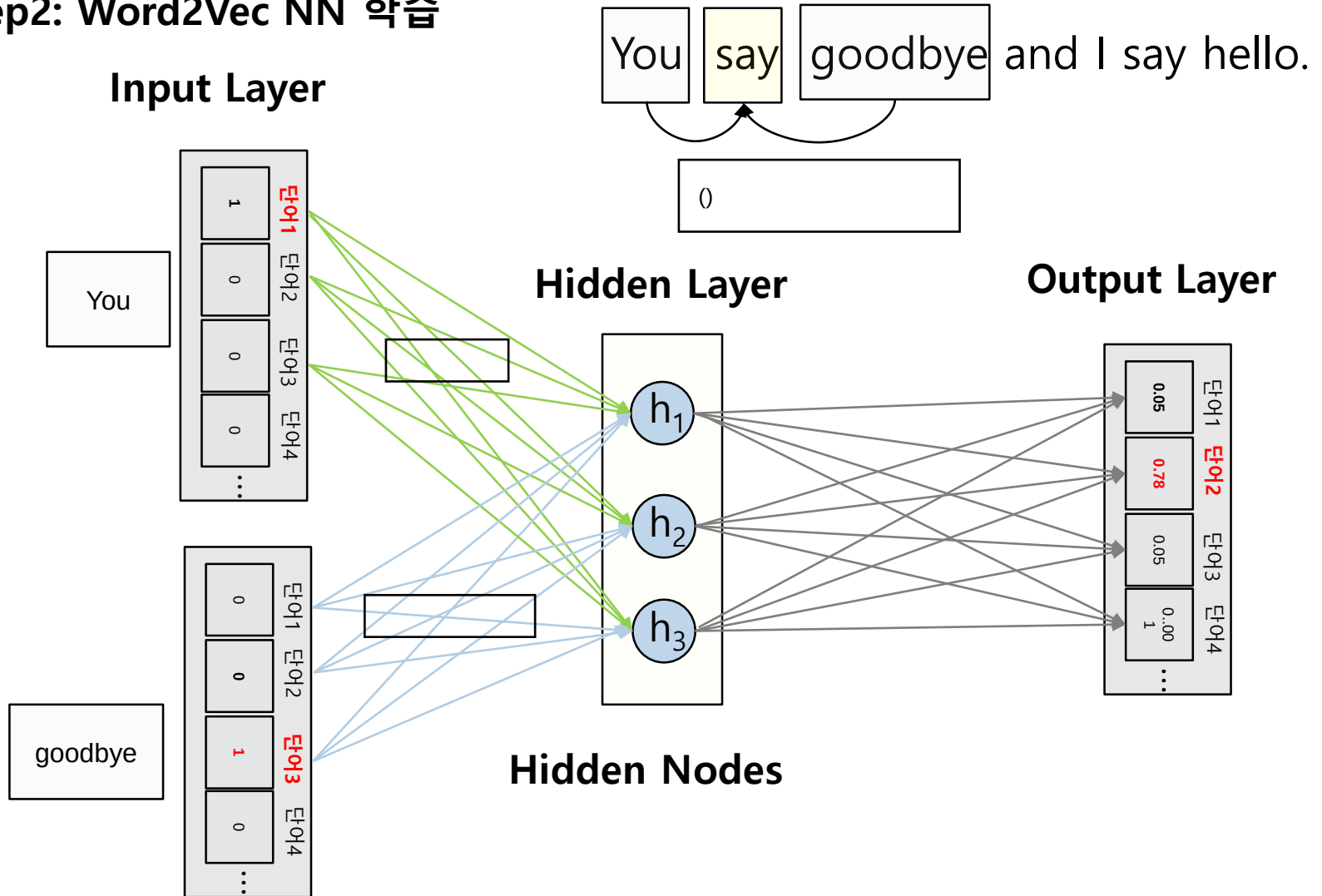
Step1: One-Hot Vector

You say goodbye and I say hello.

	단어1	단어2	단어3	단어4					
You	<table><tr><td>1</td></tr></table>	1	<table><tr><td>0</td></tr></table>	0	<table><tr><td>0</td></tr></table>	0	<table><tr><td>0</td></tr></table>	0	...
1									
0									
0									
0									
	단어1	단어2	단어3	단어4					
say	<table><tr><td>0</td></tr></table>	0	<table><tr><td>1</td></tr></table>	1	<table><tr><td>0</td></tr></table>	0	<table><tr><td>0</td></tr></table>	0	...
0									
1									
0									
0									
	단어1	단어2	단어3	단어4					
goodbye	<table><tr><td>0</td></tr></table>	0	<table><tr><td>0</td></tr></table>	0	<table><tr><td>1</td></tr></table>	1	<table><tr><td>0</td></tr></table>	0	...
0									
0									
1									
0									
	단어1	단어2	단어3	단어4					
and	<table><tr><td>0</td></tr></table>	0	<table><tr><td>0</td></tr></table>	0	<table><tr><td>0</td></tr></table>	0	<table><tr><td>1</td></tr></table>	1	...
0									
0									
0									
1									

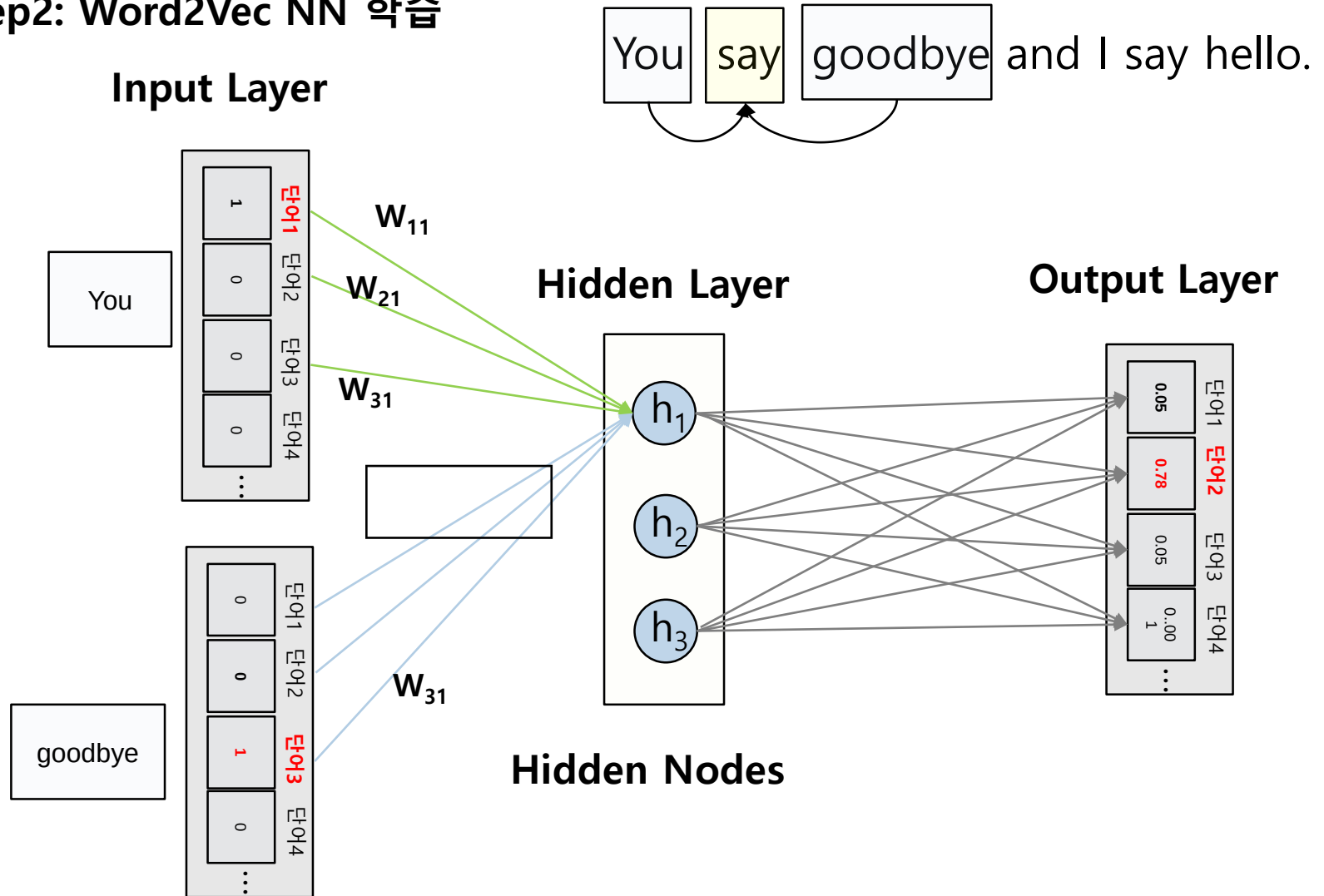
Word2Vec - CBOW

Step2: Word2Vec NN 학습



Word2Vec - CBOW

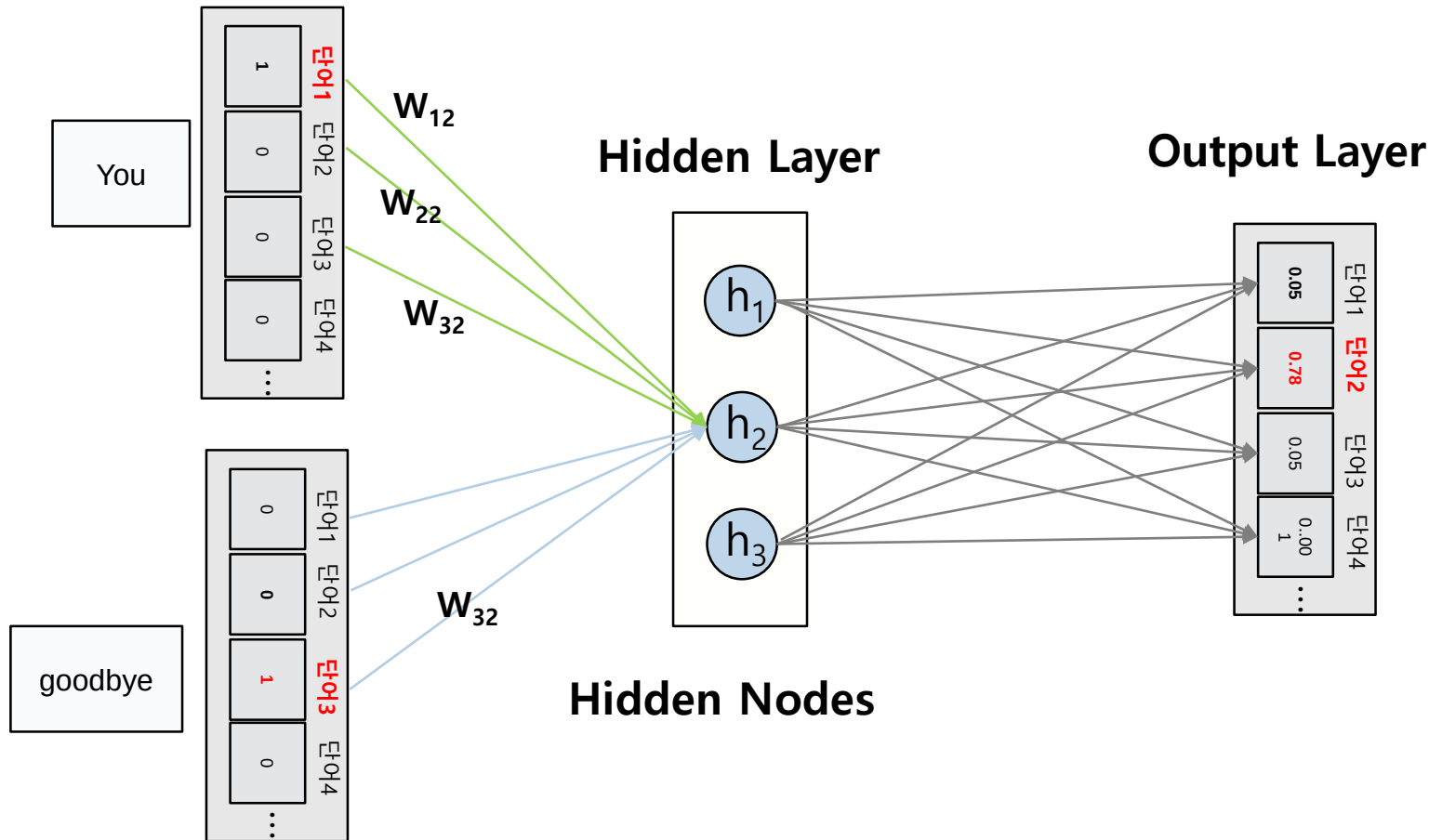
Step2: Word2Vec NN 학습



Word2Vec - CBOW

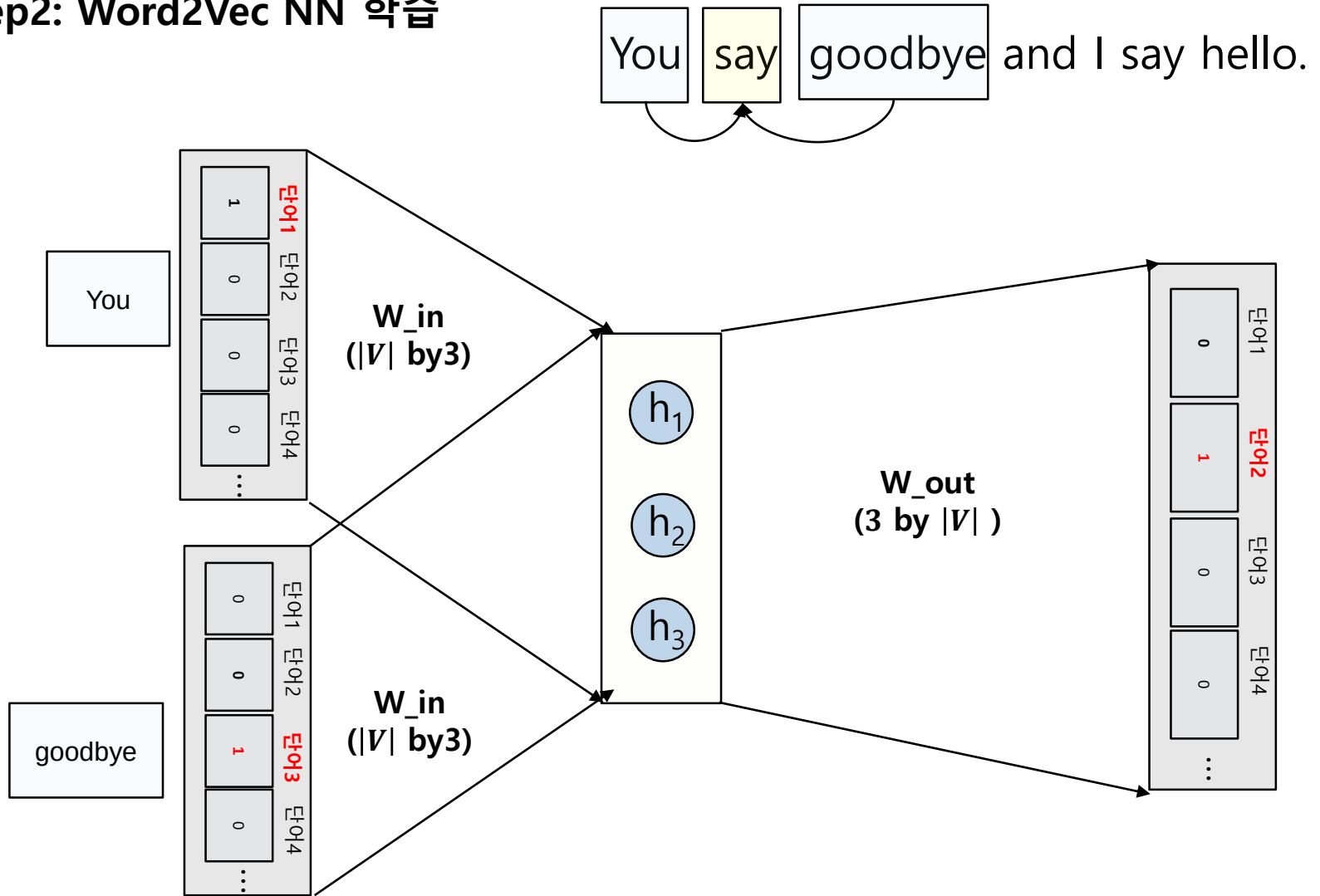
Step2: Word2Vec NN 학습

Input Layer



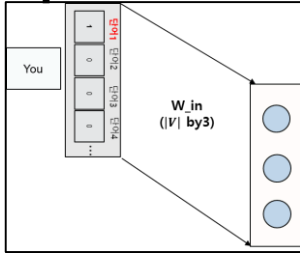
Word2Vec - CBOW

Step2: Word2Vec NN 학습



Word2Vec - CBOW

Step2: Word2Vec NN 학습



You say goodbye and I say hello.

W_{in}
($|V|$ by 3)

You

$|V|$



$\times$

$|V|$

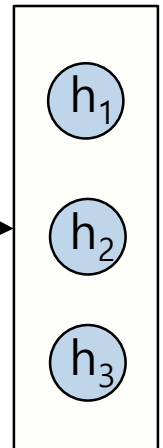
853. (W_{11})	5.2 (W_{12})	7.8 (W_{13})

3

1

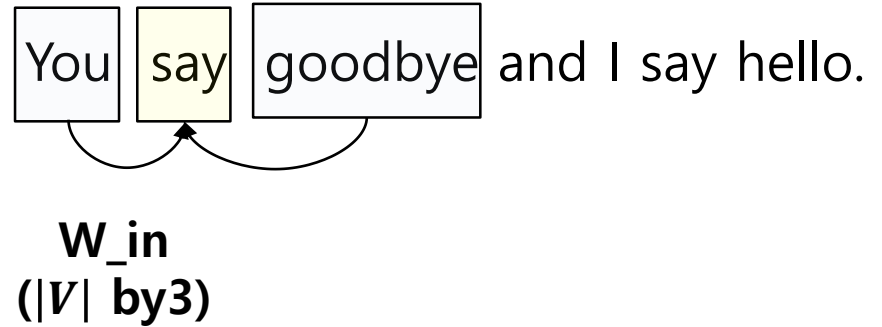
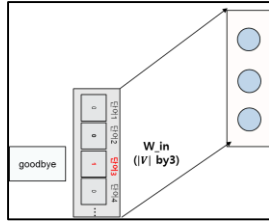
2

...



Word2Vec - CBOW

Step2: Word2Vec NN 학습



goodbye

$|V|$

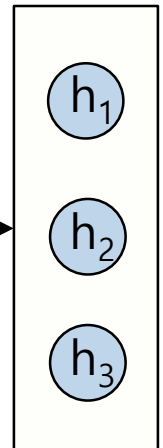


×

$|V|$

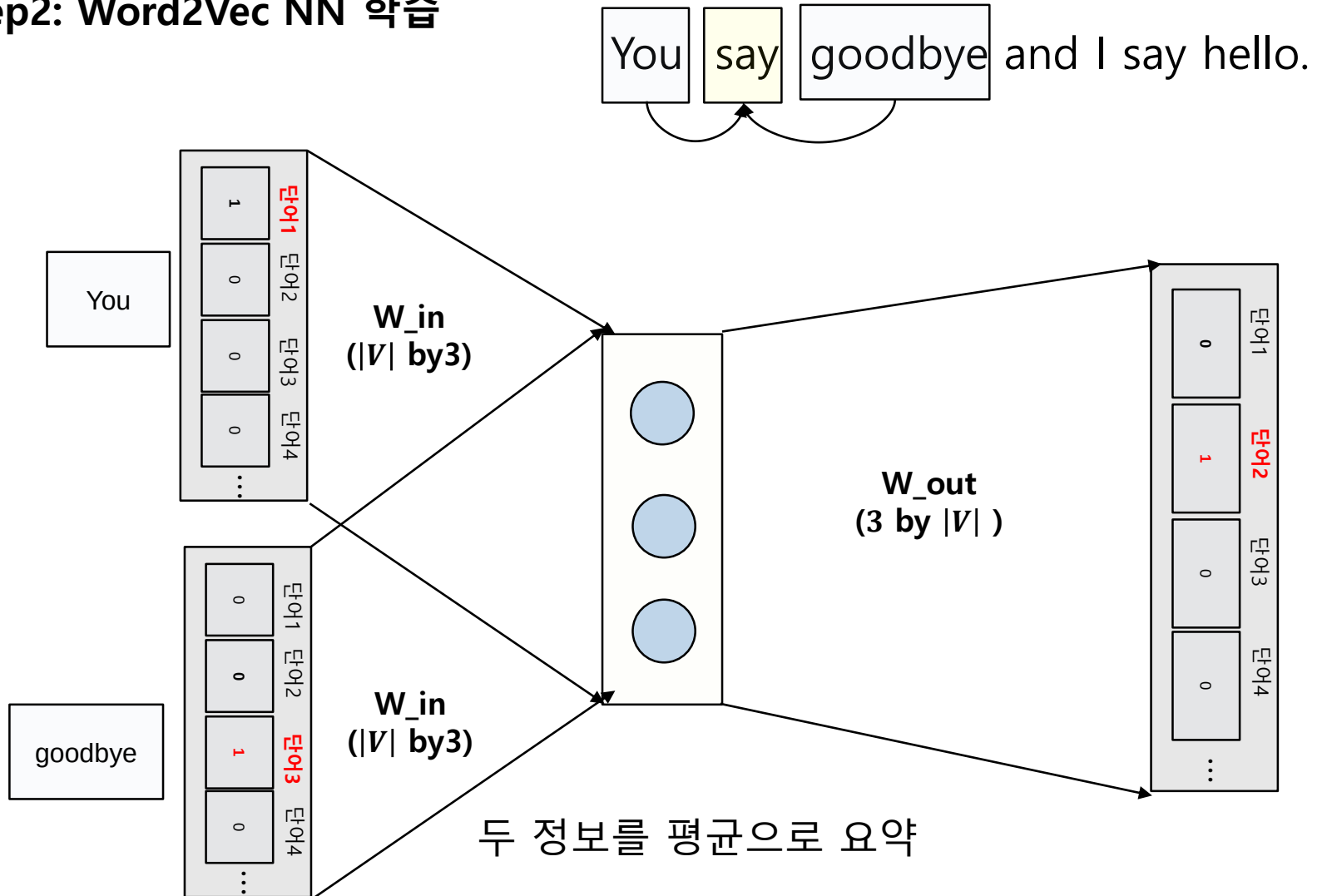
853. (W_{31})	5.2 (W_{32})	7.8 (W_{33})

3



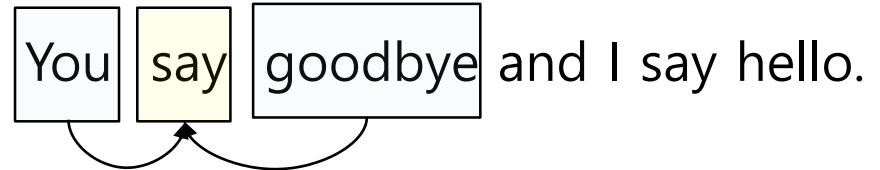
Word2Vec - CBOW

Step2: Word2Vec NN 학습

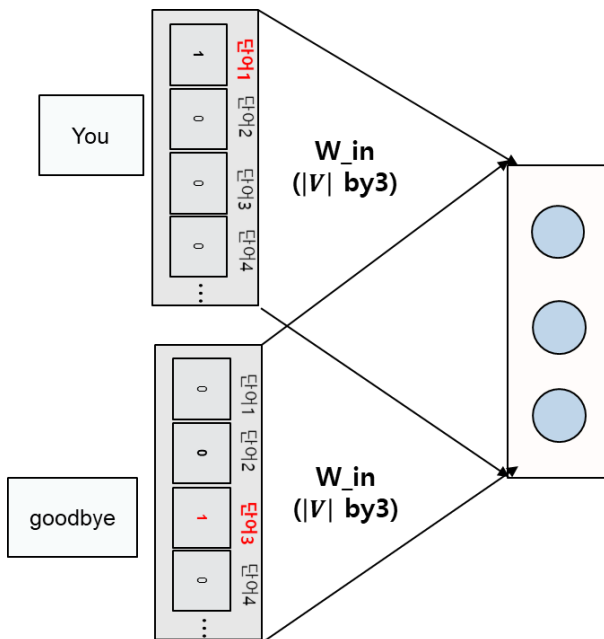


Word2Vec - CBOW

Step2: Word2Vec NN 학습



타겟 단어 주변의 단어가 고정된 크기의 벡터로 요약



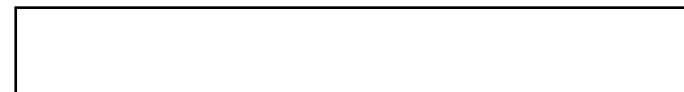
두 정보를 평균으로 요약

112.3	3.4	5.6
-------	-----	-----

853.	5.2	7.8
------	-----	-----

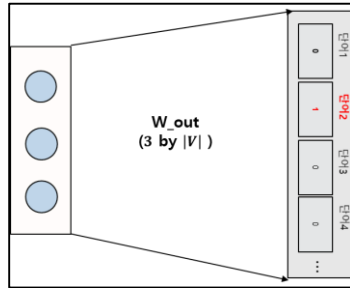
Avg ↓

XX	XXX	XX
----	-----	----



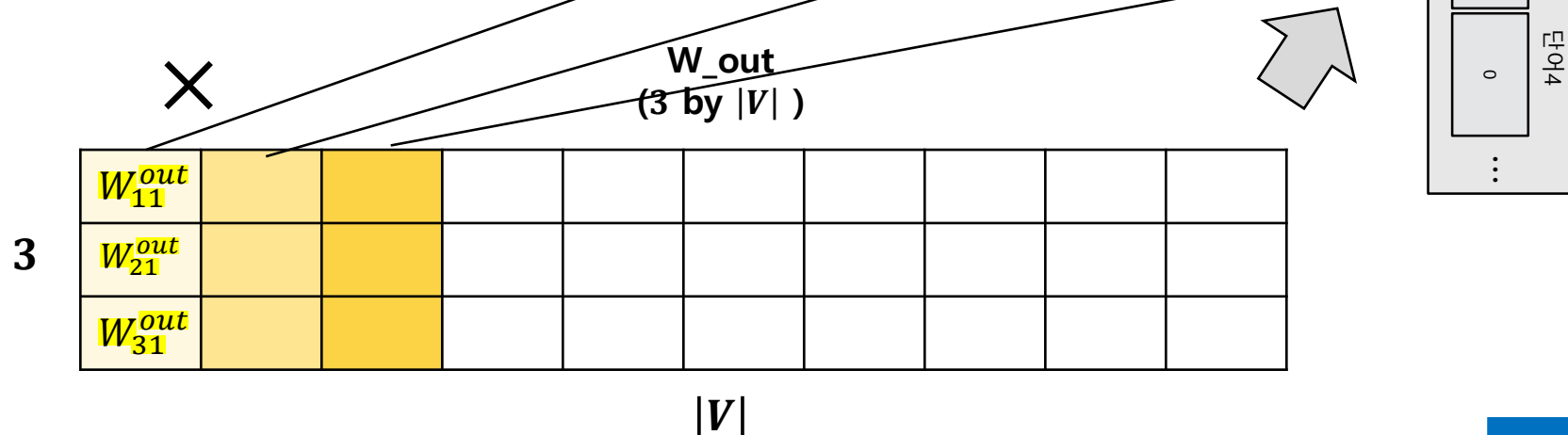
Word2Vec - CBOW

Step2: Word2Vec NN 학습

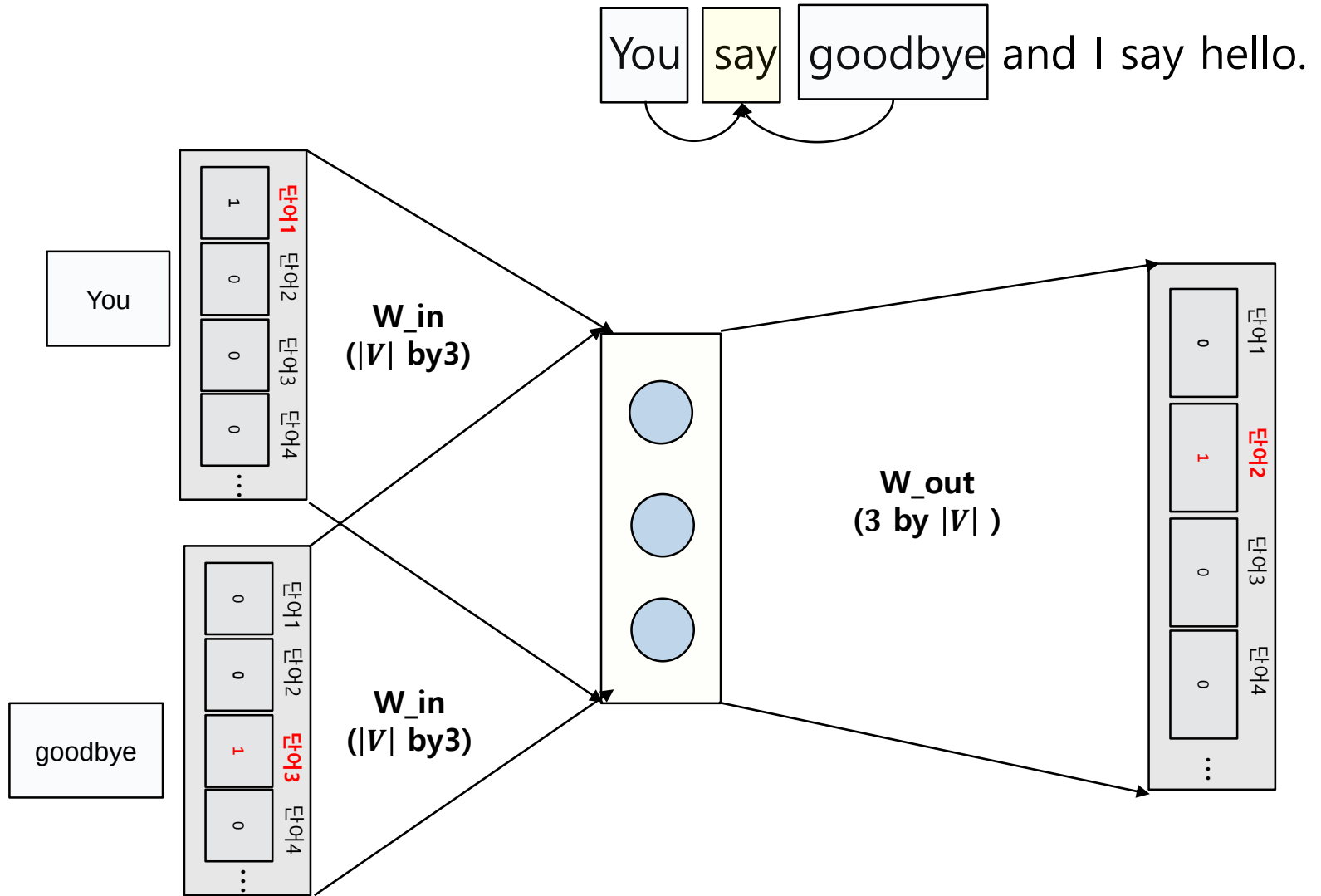


You say goodbye and I say hello.

요약된 Hidden nodes

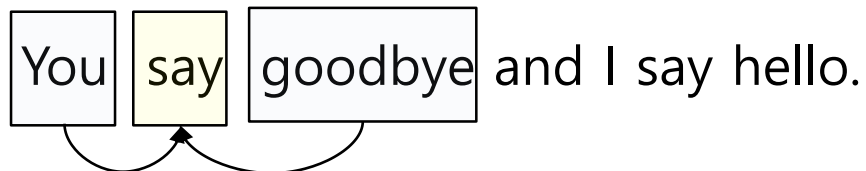


Word2Vec - CBOW



Which one should we use as Look-Up table?

Step2: Word2Vec NN 학습



W_{in}
($|V|$ by 3)

Look up Table

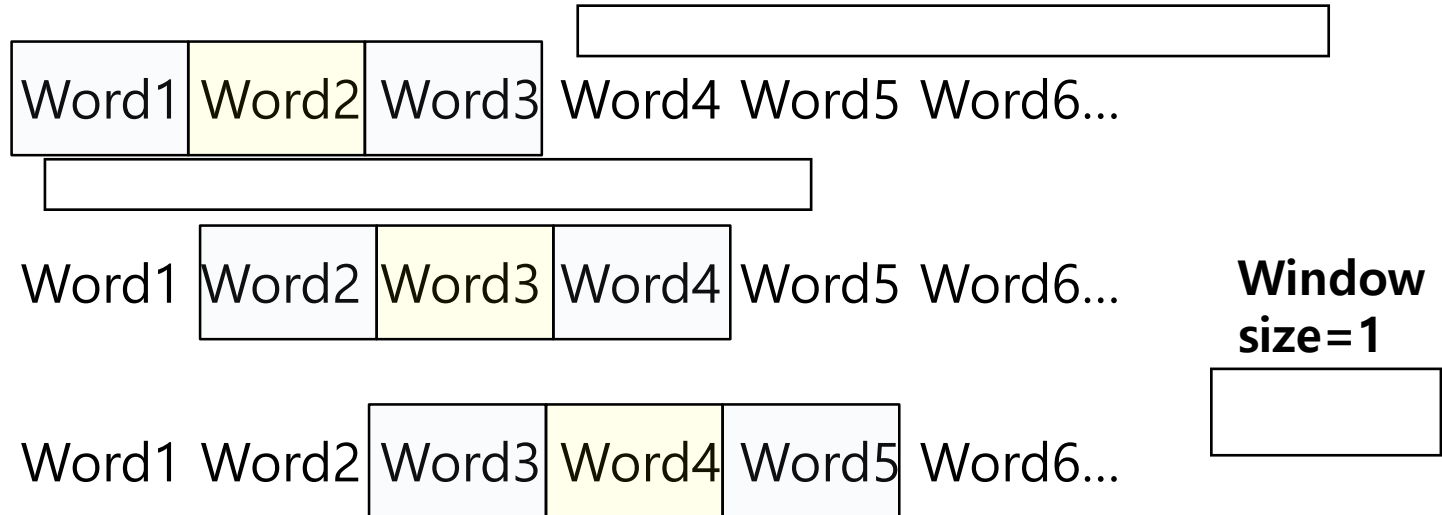
W_{out}
(3 by $|V|$)

0	단어1 단어2 단어3 단어4
0	
1	
0	
0	
0	
0	
0	

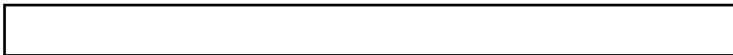
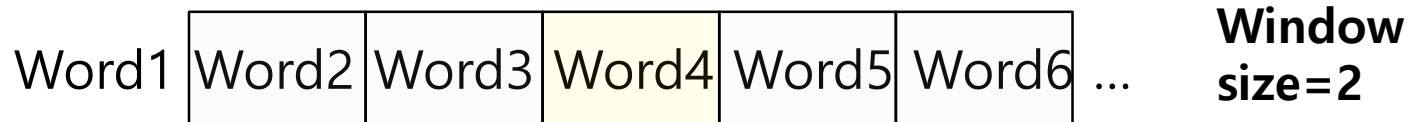
853.	5.2	7.8

Word2Vec - CBOW 하이퍼파라미터

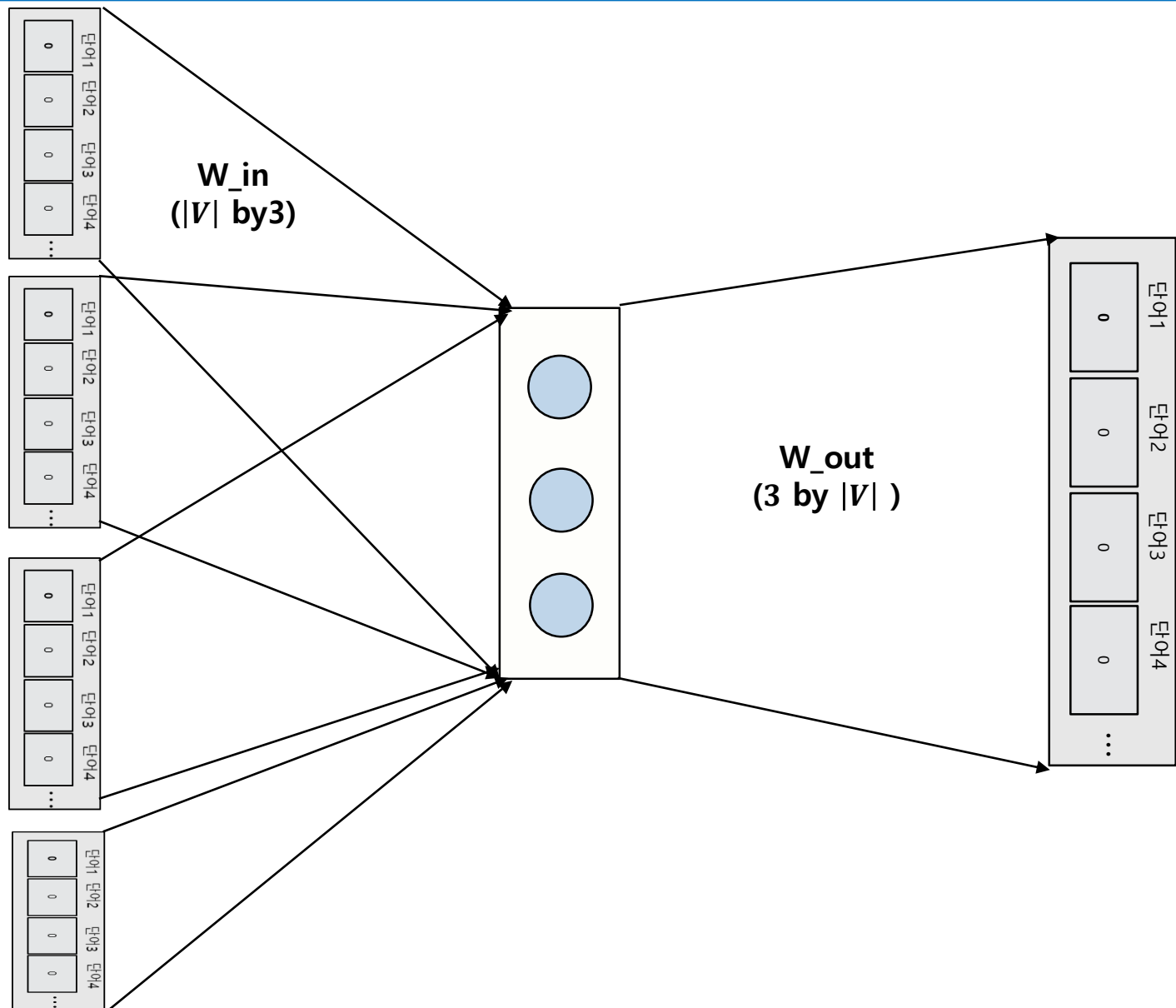
- **Sliding Window** 방식을 통해서 단어들의 문맥을 고려해서 수치화



- **Sliding Window 사이즈**를 변화시켜 문맥을 다양하게 반영



Word2Vec - CBOW 하이퍼파라미터

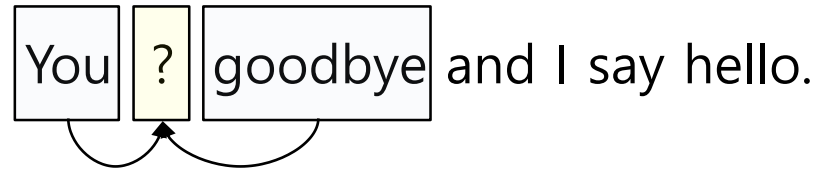


Word2Vec: Skip-gram VS CBOW

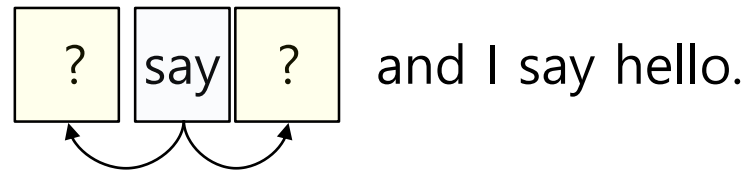
You say goodbye and I say hello.



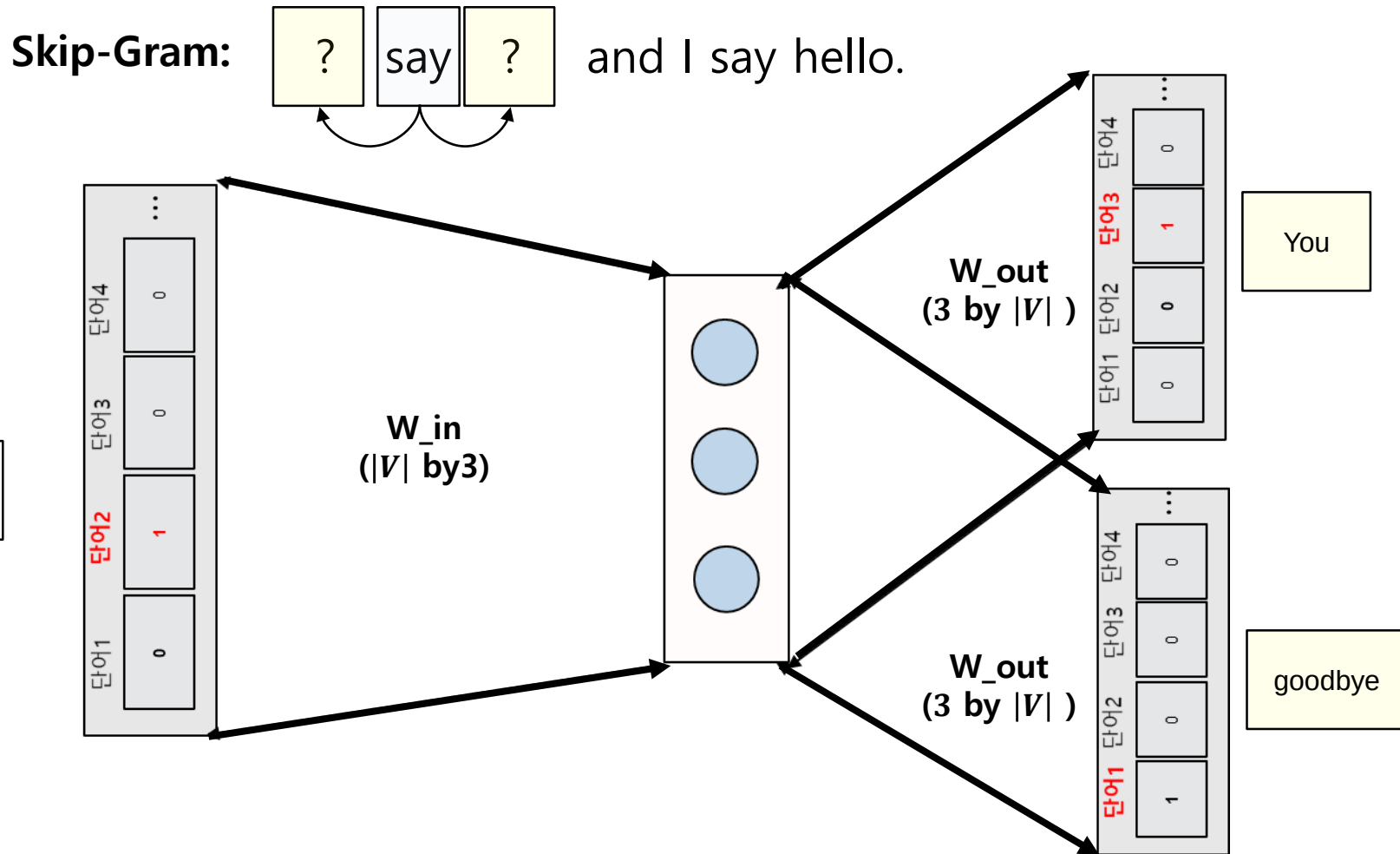
CBOW:



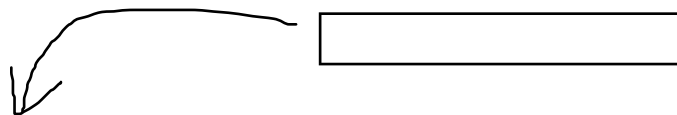
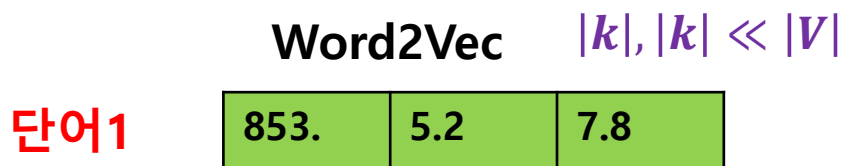
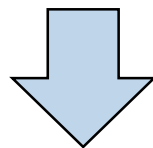
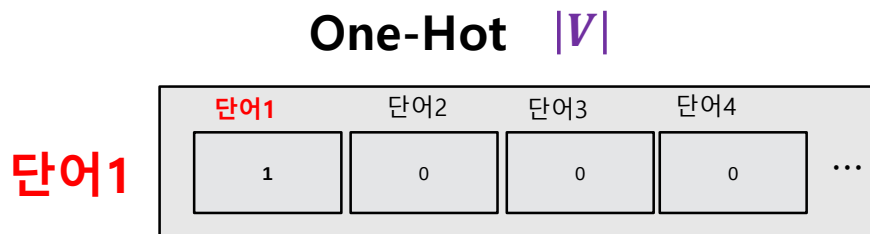
Skip-Gram:



Word2Vec: Skip-gram VS CBOW



Word2Vec - 정리



3 Not: Not High-Dim, Not Sparseness, Preserving Similarity between Words



Process is End to End, meaning simple

How to Construct Word2Vec?

1. Basic Neural Network Review

2. Word2Vec 학습 및 실습